

TYPES AND MUTABILITY

Type	Mutable	Hashable	Note
int float bool str bytes	no	yes	bool is an int
tuple	no	if items are	fixed record
frozenset	no	yes	immutable set
list	yes	no	ordered, dynamic array
dict	yes	no	insertion-ordered (3.7+)
set	yes	no	unique, unordered

- Dict keys and set members must be hashable. `None` is a singleton: test with `is None`.
- Truthiness: `0`, `0.0`, `''`, `[]`, `{}`, `set()`, `None` are falsy. `[] == False` is `False`; `bool([])` is `False`.
- `==` compares values, `is` compares identity. Small ints and interned strings may share objects: never rely on `is` for values.

NUMBERS, TRUTHINESS, FORMATTING

<code>7 // 2, -7 // 2, -7 % 2</code>	<code># 3, -4, 1 (floors)</code>
<code>0.1 + 0.2 == 0.3</code>	<code># False</code>
<code>math.isclose(0.1 + 0.2, 0.3)</code>	<code># True</code>
<code>int("3.7")</code>	<code># ValueError</code>
<code>int(float("3.7")), int(3.99)</code>	<code># 3, 3 (truncates)</code>
<code>bool("False"), bool([0])</code>	<code># True, True</code>
<code>f"{0.4567:.1%} {42:05d}"</code>	<code># '45.7% 00042'</code>
<code>f"{'a':<4} {'b':>4} {n:}"</code>	<code># align; debug</code>

- Compare floats with a tolerance. `round(2.5)` is `2`: ties go to even.
- Text from a file is truthy even when it says "False": compare `s.lower() == "true"`. Use `x is None`, not `x or d`, when `0` is valid.

COMPLEXITY OF BUILT-INS

Operation	Cost
<code>list[i]</code> , <code>append</code> , <code>pop()</code>	$O(1)$
<code>list.insert(0, x)</code> , <code>pop(0)</code> , <code>x in list</code>	$O(n)$
<code>dict/set</code> <code>get</code> , <code>set</code> , <code>in</code> , <code>delete</code>	$O(1)$ avg
<code>sorted</code> , <code>list.sort</code>	$O(n \log n)$, stable
<code>deque</code> <code>append/pop</code> both ends	$O(1)$
<code>heapq</code> <code>push / pop</code> smallest	$O(\log n)$
<code>bisect</code> search in sorted list	$O(\log n)$
<code>s += t</code> in a loop	$O(n^2)$: use <code>''.join(parts)</code>
<code>slice a[i:j]</code>	$O(j - i)$ copy

- Amortised $O(1)$ `append`: the array over-allocates and doubles.

SYNTAX YOU MUST KNOW COLD

<code>a, *mid, z = [1, 2, 3, 4]</code>	<code># unpacking</code>
<code>a, b = b, a</code>	<code># swap</code>
<code>x[::-1]; x[1:-1]; x[::2]</code>	<code># slices</code>
<code>[f(i) for i in r if i % 2]</code>	<code># list comp</code>
<code>{k: v for k, v in pairs}</code>	<code># dict comp</code>
<code>(i * i for i in r)</code>	<code># generator</code>
<code>for i, v in enumerate(xs, 1): ...</code>	
<code>for a, b in zip(xs, ys): ...</code>	<code># shortest</code>
<code>sorted(xs, key=lambda t: (-t[1], t[0]))</code>	
<code>max(xs, key=len); any(...); all(...)</code>	
<code>y = a if cond else b</code>	
<code>if (n := len(xs)) > 3: ...</code>	<code># walrus</code>
<code>f"{x:.2f} {n:,} {s!r:>8}"</code>	<code># f-string</code>

- `sorted` returns a new list; `.sort()` sorts in place and returns `None`.
- `dict.get(k, d)`, `setdefault`, `d | e` merge (3.9+), `{**a, **b}`.

METHODS WORTH REMEMBERING

- list**: `append` `extend` `insert` `remove` `pop` `index` `count` `sort` `reverse` `clear`; `xs + ys` copies; `xs * 3` repeats.
- dict**: keys values items `get` `pop` `popitem` `update` `setdefault`; iterate `for k, v in d.items()`.
- set**: `add` `discard` `remove` `union` `|` `intersection` & `difference` - `symmetric_difference` `^`, `issubset`; `x in s` is $O(1)$.
- Numbers**: `divmod(a, b)`, `round(x, 2)` (banker's rounding), `abs` `pow` `sum` `min` `max`, `int("12")`, `float`, `math.sqrt` `floor` `ceil` `log` `gcd` `inf`.
- `range(a, b, step)` is lazy; `list(range(5)).reversed()`, `sorted(reverse=True)`, `zip(*matrix)` transposes.

- Chained comparison: `0 < x < 10`. and/or return an operand: `x or default`.

FUNCTIONS AND SCOPE

```
def f(a, b=2, *args, c, **kw): ...
def f(x, /, y, *, z): ...      # pos-only / kw-only
lambda x: x * 2
nonlocal n                    # rebind enclosing variable
```

- LEGB** lookup: Local, Enclosing, Global, Built-in.
- Arguments are passed by object reference: mutating a passed list changes the caller's list; rebinding the name does not.

TRAP Mutable default argument: `def f(x, acc=[])` shares one list across calls. Use `acc=None` then `acc = acc or []`. **Late binding closures:** `[lambda: i for i in range(3)]` all return `2`. Bind with `lambda i=i: i`.

- Decorator = function that takes a function and returns a function; use `functools.wraps`.
- Generator = function with `yield`; lazy, one pass, constant memory. `yield` from delegates.

LOOPS AND ITERATORS

```
for x in xs:
    if ok(x): break
else: ...
it = iter(xs); next(it, None) # default: no error
next(iter(rows), None)      # peek at first item
list(islice(gen(), 10))     # first 10 of a stream
zip(a, b, strict=True)      # error if lengths differ
ks, vs = zip(*pairs)        # unzip
for blk in iter(lambda: f.read(4096), b''):
```

- A `for` loop calls `next()` until `StopIteration`. A generator is spent after one pass: `list(g)` twice gives `[]` the second time.
- Laziness saves memory, not time. Batch pattern: fill a list to size `n`, `yield` it, `reset`, then `yield` the leftover.

COPYING AND EQUALITY

- Assignment never copies. `b = a` is the same object.
- Shallow copy (`a.copy()`, `a[:]`, `list(a)`, `copy.copy`) copies the container, shares the items. `copy.deepcopy` copies recursively.
- `[[0] * 3] * 3` makes three references to one row. Use `[[0] * 3 for _ in range(3)]`.
- Strings are immutable; `s[0] = 'a'` raises `TypeError`.

OOP AND SPECIAL METHODS

- `__init__` initialises; `__new__` creates. `__repr__` for developers (unambiguous), `__str__` for users.
- Operators: `__eq__` `__lt__` `__hash__` `__len__` `__getitem__` `__iter__` `__next__` `__contains__` `__call__` `__enter__` `__exit__`.
- Defining `__eq__` without `__hash__` makes the class unhashable.
- @property computed attribute; @classmethod gets `cls` (alt constructors); @staticmethod gets nothing.
- Inheritance: `super().__init__()`; method resolution order via `C3` (`Cls.__mro__`).
- @dataclass generates `__init__`, `__repr__`, `__eq__`; `frozen=True` for immutable. `__slots__` saves memory.
- Duck typing: "if it quacks"; use ABCs or `Protocol` for contracts.

ERRORS, CONTEXT MANAGERS

```
try:
    risky()
except (KeyError, ValueError) as e:
    handle(e)
else:
    ok() # no exception
finally:
    cleanup() # always
with open(p) as f: ... # closes on exit
raise ValueError("msg") from e
```

Catch specific exceptions, never bare `except:`. Custom errors: subclass `Exception`.

COLLECTIONS

```

from collections import (Counter, defaultdict,
                        deque, namedtuple)

Counter("aab").most_common(1)      # [('a', 2)]
d = defaultdict(list); d[k].append(v)
q = deque(maxlen=3); q.appendleft(x)
q.popleft()                        # 0(1) queue
P = namedtuple("P", "x y")

```

ITERTOOLS, FUNCTOOLS, HEAPQ, BISECT

```

from itertools import (permutations, combinations,
                      product, accumulate, groupby, chain, islice)
list(combinations("abc", 2))       # ab ac bc
list(accumulate([1, 2, 3]))       # [1, 3, 6]
import heapq
h = []; heapq.heappush(h, x); heapq.heappop(h)
heapq.nlargest(3, xs)              # top-k
heapq.heappush(h, -x)              # max-heap trick
import bisect; bisect.bisect_left(xs, x)
from functools import lru_cache, partial, reduce
@lru_cache(maxsize=None)          # memoise

```

groupby needs sorted input (it groups neighbours only). heapq is a min-heap only. chain(a, b) joins iterables; product(a, b) gives every pairing (a parameter grid); islice slices any iterator.

STRINGS

- split(), ' '.join(xs), strip lstrip rstrip, lower upper title, startswith endswith, find (-1) vs index (error), replace, count, isalpha isdigit isalnum, zfill, partition.
- ord('a') = 97, chr(97). Reverse s[::-1]. Palindrome: s == s[::-1].
- Anagram: sorted(a) == sorted(b) or Counter(a) == Counter(b).
- Regex: re.findall(r"\d+", s), re.sub(r"\s+", " ", s), re.match anchors at start, re.search anywhere; groups (...), lazy .*?.

CLASSIC SNIPPETS

```

# two-sum in O(n)
seen = {}
for i, x in enumerate(nums):
    if t - x in seen: return seen[t - x], i
    seen[x] = i
# dedupe, keep order
list(dict.fromkeys(xs))
# first non-repeating char
c = Counter(s)
next((ch for ch in s if c[ch] == 1), None)
# max subarray (Kadane)
best = cur = a[0]
for x in a[1:]:
    cur = max(x, cur + x); best = max(best, cur)
# flatten one level
[y for x in xs for y in x]
# word frequency
Counter(text.lower().split())

```

FROM SCRATCH: STATS AND ML

```

import math
def mean(x): return sum(x) / len(x)
def var(x, ddof=1):
    m = mean(x)
    return sum((v - m) ** 2 for v in x) / (len(x) - ddof)
def std(x): return math.sqrt(var(x))
def sigmoid(z): return 1 / (1 + math.exp(-z))
def softmax(z):
    m = max(z); e = [math.exp(v - m) for v in z]
    return [v / sum(e) for v in e]
def dist(a, b):
    return math.sqrt(sum((p - q) ** 2 for p, q in zip(a, b)))

```

Sample variance divides by n - 1. Subtract the max in softmax for numerical stability.

FROM SCRATCH: GRADIENT DESCENT

```

# fit y = w*x + b by gradient descent
w = b = 0.0; lr = 0.01
for _ in range(1000):
    gw = gb = 0.0
    for x, y in data:
        err = (w * x + b) - y
        gw += 2 * err * x / len(data)
        gb += 2 * err / len(data)
    w -= lr * gw; b -= lr * gb

```

K-means: assign each point to the nearest centre, move each centre to its group's mean, repeat. KNN: sort by distance, majority vote of the k nearest.

FILES, JSON, DATES

```

from pathlib import Path; import json
txt = Path("a.txt").read_text()
data = json.loads(s); json.dumps(d, indent=2)
from datetime import datetime, timedelta
datetime.strptime("2025-01-31", "%Y-%m-%d")
d + timedelta(days=7); d.strftime("%b %d")
(d2 - d1).days; d.weekday()      # Monday = 0
p = Path("data") / "raw" / "a.csv"
p.name, p.stem, p.suffix, p.parent
p.with_suffix(".parquet"); p.exists()
Path("out").mkdir(parents=True, exist_ok=True)
with open(a) as src, open(b, "w") as dst: ...

```

```

from contextlib import contextmanager
@contextmanager
def timer():
    t = time.perf_counter()
    try: yield          # with-body runs here
    finally: print(time.perf_counter() - t)

```

Use with open(...); csv.DictReader for CSV; pd.read_csv for real work.

CONCURRENCY AND MEMORY

- GIL: one thread runs Python bytecode at a time. Threads help I/O-bound work; use multiprocessing / ProcessPoolExecutor for CPU-bound; asyncio for many network waits.
- Memory: reference counting plus a cycle collector. Generators and iterators avoid building big lists.
- Iterator protocol: __iter__ returns self, __next__ raises StopIteration.
- Profiling: timeit, cProfile; vectorise with NumPy before optimising loops.

GOTCHAS

TRAP sum of floats: 0.1 + 0.2 ≠ 0.3; use math.isclose. Modifying a list while iterating over it skips items; iterate over a copy. Integer division // floors toward -inf: -7 // 2 = -4. % has the sign of the divisor. dict order is insertion order; set order is arbitrary. is on ints/strings; except Exception hiding bugs; = on floats.

ERROR MESSAGES -> CAUSE

Message	Usual cause
TypeError: unhashable type: 'list'	list used as dict key or set item
KeyError / IndexError	missing key / index out of range
UnboundLocalError	assigned in a function before use, needs nonlocal/global
RecursionError	no base case (default limit ~1000)
ValueError: too many values to unpack	length mismatch in unpacking
AttributeError: 'NoneType'...	a function returned None (e.g. .sort())

STYLE AND TOOLING

- PEP 8, type hints (def f(x: int) → str), docstrings, small pure functions, f-strings.
- venv + requirements.txt; pytest for tests (assert f(2) == 4); black/ruff for formatting.
- if __name__ == "__main__": guards script code.

SAY IT IN 20 SECONDS

- list vs tuple: mutable vs immutable, tuples are hashable and slightly faster.
- Shallow vs deep copy: shared inner objects vs fully independent.
- Generator vs list: lazy and one-pass vs materialised in memory.
- GIL: one bytecode thread at a time; threads for I/O, processes for CPU.
- Decorator: a function that wraps a function to add behaviour.
- "State complexity for every solution."

LOGICAL EXECUTION ORDER

- FROM/JOIN → WHERE → GROUP BY → HAVING → SELECT (aliases, window functions) → DISTINCT → ORDER BY → LIMIT/OFFSET
- SELECT aliases are **not** visible in WHERE/HAVING (they are in ORDER BY).
 - Window functions cannot be in WHERE: compute in a CTE or subquery, filter outside.
 - WHERE filters rows before grouping; HAVING filters groups after aggregation.
 - This is the logical order; the optimiser may execute differently but never changes the result.

QUERY TEMPLATE

```
WITH x AS (SELECT ... FROM t WHERE ...)
SELECT a, COUNT(*) AS n, AVG(v) AS m
FROM x JOIN y ON x.id = y.id
WHERE d ≥ '2025-01-01' AND d < '2025-02-01'
GROUP BY a
HAVING COUNT(*) > 5
ORDER BY n DESC
LIMIT 10;
```

Dates: use half-open ranges ≥ start AND < next, never BETWEEN on timestamps (it misses most of the last day).

JOINS

Join	Keeps
INNER	matches only
LEFT	all left rows; NULLs on the right if no match
RIGHT / FULL	all right / all rows of both
CROSS	every pair (m × n rows)
SELF	table joined to itself (manager, previous row)

- Anti-join** (no match): LEFT JOIN ... WHERE r.id IS NULL, or NOT EXISTS.
- Semi-join** (has a match, no duplication): EXISTS.
- Right-table filter in WHERE after a LEFT JOIN turns it into an inner join: put it in ON.
- Duplicate keys on either side multiply rows (**fan-out**). Check the grain of both tables first.
- USING (id) merges equal-named key columns; NATURAL JOIN is risky, avoid.

NULL RULES

- NULL = NULL is unknown, not true. Use IS NULL / IS NOT NULL.
- COUNT(*) counts rows; COUNT(col) skips NULLs; COUNT(DISTINCT col) unique non-NULLs.
- SUM/AVG/MIN/MAX ignore NULLs: AVG of 1, 2, NULL is 1.5.
- NULL + 1 is NULL. COALESCE(x, 0) = first non-NULL. NULLIF(a, 0) = NULL if equal (safe divide).
- ORDER BY NULL placement differs by engine; use NULLS FIRST/LAST (PG).
- NULL in WHERE a <> 1 is filtered out: NULL rows vanish silently.
- TRAP** NOT IN (subquery) returns nothing when the subquery contains a NULL. Use NOT EXISTS.

AGGREGATION AND SETS

- Aggregates: COUNT SUM AVG MIN MAX, STRING_AGG / GROUP_CONCAT, PERCENTILE_CONT(.5) WITHIN GROUP (ORDER BY x).
- Every non-aggregated SELECT column must be in GROUP BY.
- Conditional counts: SUM(CASE WHEN s = 'x' THEN 1 ELSE 0 END) or COUNT(*) FILTER (WHERE s = 'x') (PG).
- GROUP BY multiple columns; ROLLUP/CUBE for subtotals; GROUPING SETS.
- UNION stacks and removes duplicates; UNION ALL keeps them (faster); INTERSECT, EXCEPT. Same column count and types.
- IN (list) vs EXISTS: EXISTS stops at the first match and is NULL-safe.
- Subquery types: scalar (one value), row/table, correlated (refers to the outer row, runs per row).

- CTE (WITH) = named, readable subquery. Recursive CTE = anchor UNION ALL recursive step.

WINDOW FUNCTIONS

```
fn(x) OVER (
  PARTITION BY g
  ORDER BY d
  ROWS BETWEEN 6 PRECEDING AND CURRENT ROW)
```

- Keeps every row (unlike GROUP BY). PARTITION BY = groups; ORDER BY = order within the group.
- Default frame when ORDER BY is present: RANGE UNBOUNDED PRECEDING ... CURRENT ROW. LAST_VALUE needs ... UNBOUNDED FOLLOWING.
- Frames count **rows**, so missing days need a calendar table.

Function	For 90, 70, 70, 50
ROW_NUMBER	1, 2, 3, 4 (ties arbitrary)
RANK	1, 2, 2, 4 (gaps)
DENSE_RANK	1, 2, 2, 3 (no gaps)
NTILE(4)	quartile bucket 1-4

- LAG(x, n) / LEAD(x, n): n rows back / forward. FIRST_VALUE, NTH_VALUE. SUM() OVER (ORDER BY d) = running total. PERCENT_RANK, CUME_DIST for percentiles.
- Window functions run after GROUP BY: SUM(SUM(x)) OVER () is valid.

HANDY FUNCTIONS

- Strings: LOWER TRIM SUBSTR LENGTH CONCAT / ||, LIKE 'a%', REPLACE, SPLIT_PART, REGEXP_REPLACE, POSITION.
- Dates (PG): DATE_TRUNC('month', d), EXTRACT(dow FROM d), d + INTERVAL '7 day', AGE(a, b), CURRENT_DATE.
- MySQL: DATE_FORMAT, DATEDIFF(a, b), DATE_ADD. BigQuery: DATE_DIFF, TIMESTAMP_TRUNC. SQL Server: DATEDIFF(day, a, b), TOP n.
- Cast: CAST(x AS INT), x::numeric (PG). Integer division truncates: write 100.0 * a / b.
- CASE WHEN ... THEN ... ELSE ... END works in SELECT, ORDER BY and inside aggregates.
- Limit rows: LIMIT n OFFSET k (PG/MySQL), FETCH FIRST n ROWS, TOP n.
- DISTINCT ON (k) (PG): first row per k by ORDER BY.

DATE RECIPES (POSTGRESQL)

- Month start: DATE_TRUNC('month', d); next month: + INTERVAL '1 month'.
- Last 7 days: d ≥ CURRENT_DATE - 7.
- Days between: b - a (dates) or EXTRACT(EPOCH FROM b - a) / 86400.
- Weekday (0 = Sunday): EXTRACT(dow FROM d); ISO week: EXTRACT(week FROM d).
- Hour bucket: DATE_TRUNC('hour', ts).
- Time zones: store UTC, convert with ts AT TIME ZONE 'Asia/Kolkata'.
- Calendar table: generate_series(d1, d2, '1 day') fills missing days before a moving average.

INTERVIEW QUESTION -> TOOL

Asked	Reach for
Nth highest, top N per group	DENSE_RANK / ROW_NUMBER in a CTE
Never did X / missing rows	anti-join, NOT EXISTS
Duplicates	GROUP BY ... HAVING COUNT(*) > 1
Change vs previous	LAG
Running or moving total	SUM/AVG OVER + frame
Consecutive days	date - ROW_NUMBER
Retention, cohorts	first-date CTE + LEFT JOIN
Funnel	COUNT(DISTINCT CASE ...)
Rows to columns	SUM(CASE ...) pivot
Hierarchy, org chart	recursive CTE
Median	PERCENTILE_CONT or row-number trick

TOP-N PER GROUP, NTH HIGHEST

```
SELECT * FROM (
  SELECT *, DENSE_RANK() OVER (
    PARTITION BY dept ORDER BY sal DESC) r
  FROM emp) t
WHERE r ≤ 3;
-- second highest without windows
SELECT MAX(sal) FROM emp
WHERE sal < (SELECT MAX(sal) FROM emp);
```

Say the tie rule: DENSE_RANK keeps all ties, ROW_NUMBER returns exactly N (add a tie-breaker). Employees paid more than their manager: self-join e.boss = m.id.

DEDUPE, ANTI-JOIN, DUPLICATES

```
-- latest row per user
SELECT * FROM (SELECT *, ROW_NUMBER() OVER
  (PARTITION BY user_id ORDER BY ts DESC) rn
  FROM ev) t WHERE rn = 1;
-- emails that appear twice or more
SELECT email, COUNT(*) FROM u
GROUP BY email HAVING COUNT(*) > 1;
-- users who never ordered
SELECT u.* FROM u WHERE NOT EXISTS
  (SELECT 1 FROM o WHERE o.uid = u.id);
```

Test a DELETE as a SELECT first.

RUNNING TOTAL, MOVING AVERAGE, GROWTH

```
SUM(rev) OVER (ORDER BY d)
AVG(rev) OVER (ORDER BY d ROWS 6 PRECEDING)
rev - LAG(rev) OVER (ORDER BY m)
100.0 * (rev - LAG(rev) OVER (ORDER BY m))
  / NULLIF(LAG(rev) OVER (ORDER BY m), 0)
d - LAG(d) OVER (PARTITION BY uid ORDER BY d)
```

Lines: running total, 7-row average, change, % growth, days between orders. Aggregate to one row per day first, then window. Partition LAG by entity; the first row has NULL.

GAPS AND ISLANDS (STREAKS)

```
-- one row per user per day first (DISTINCT)
SELECT uid, MIN(d) AS start, COUNT(*) AS len
FROM (SELECT uid, d, d - (ROW_NUMBER() OVER
  (PARTITION BY uid ORDER BY d))::int AS grp
  FROM days) t
GROUP BY uid, grp
HAVING COUNT(*) ≥ 3;
```

Consecutive days minus a running counter give a constant: the streak id (PG: date - int is a date). Duplicate days break it.

RETENTION AND COHORTS

```
WITH c AS (SELECT uid, MIN(d) AS d0
  FROM logins GROUP BY uid)
SELECT c.d0, COUNT(DISTINCT c.uid) AS n,
  COUNT(DISTINCT l.uid) * 1.0
  / COUNT(DISTINCT c.uid) AS d7
FROM c LEFT JOIN logins l
  ON l.uid = c.uid AND l.d = c.d0 + 7
GROUP BY c.d0;
```

Cohort = first activity date. Denominator = cohort size, not login rows. LEFT JOIN keeps non-returns. Monthly cohorts: truncate dates to month and join on month offset.

FUNNEL AND SESSIONS

```
SELECT
  COUNT(DISTINCT uid) FILTER (WHERE e='view') v,
  COUNT(DISTINCT uid) FILTER (WHERE e='cart') c,
  COUNT(DISTINCT uid) FILTER (WHERE e='buy') b
FROM ev;
-- flag a new session after a 30 minute gap
WITH g AS (SELECT *, ts - LAG(ts) OVER
  (PARTITION BY uid ORDER BY ts) AS gap FROM ev)
SELECT *, SUM(CASE WHEN gap IS NULL
  OR gap > INTERVAL '30 min' THEN 1 ELSE 0 END)
  OVER (PARTITION BY uid ORDER BY ts) AS sess
FROM g;
```

Count users, not events (portable form: COUNT(DISTINCT CASE WHEN e='buy' THEN uid END)). A strict funnel needs step order: first timestamps must increase view < cart < buy.

PIVOT, SHARE OF TOTAL, MEDIAN

```
SELECT m,
  SUM(CASE WHEN cat='A' THEN rev ELSE 0 END) a,
  SUM(CASE WHEN cat='B' THEN rev ELSE 0 END) b
FROM s GROUP BY m;
100.0 * SUM(rev) / SUM(SUM(rev)) OVER ()
PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY x)
```

Lines: pivot, percent of total, median (PG). Portable median: number rows, take the middle one or average the two middle ones. Pivot columns are fixed in the query.

RECURSIVE CTE, "IN EVERY MONTH"

```
WITH RECURSIVE t AS (
  SELECT id, boss, 1 AS lvl
  FROM emp WHERE boss IS NULL
  UNION ALL
  SELECT e.id, e.boss, t.lvl + 1
  FROM emp e JOIN t ON e.boss = t.id)
SELECT * FROM t;
```

"Did X in all 3 months": GROUP BY uid HAVING COUNT(DISTINCT month) = 3. Churn candidates: last order older than a fixed as-of date. Guard against cycles.

TRAPS CHECKLIST

- **Fan-out**: summing an order-level column after joining items inflates totals. Aggregate before joining.
- **Right-table filter** in WHERE after LEFT JOIN drops unmatched rows. Use ON.
- **Integer division** truncates; BETWEEN on timestamps; COUNT(*) where you need COUNT(DISTINCT user).
- **Ties** in ranking; NOT IN with NULLs; LAST_VALUE default frame; GROUP BY column missing.
- Sanity check after every join: row count and totals reconcile with the source.

SPEED, INDEXES, CONCEPTS

- **Index** = sorted structure (B-tree) for lookups, ranges and joins; costs write speed and space. Composite index: leftmost column first.
- **Sargable**: no function on an indexed column (WHERE YEAR(d) = 2025 is bad, a date range is good). Select only needed columns, filter early, read EXPLAIN.
- DELETE removes rows (rollback ok); TRUNCATE empties fast; DROP removes the table.
- **ACID**: Atomic, Consistent, Isolated, Durable. **OLTP** many small writes; **OLAP** big scans (columnar warehouse).
- Keys: primary (unique, not null), foreign (references). Normalisation cuts redundancy; analytics often uses a star schema.
- Isolation anomalies: dirty read, non-repeatable read, phantom read.
- Clustered index = physical order (one per table); non-clustered = separate pointer structure.
- Large joins: filter and aggregate first; temp table if an expensive result is reused.

LIVE-ROUND ROUTINE

1. Restate the question and the **output grain** (one row per what?).
2. Inspect keys, NULLs, duplicates, date types; ask about ties and cancelled rows.
3. Plan in steps aloud: CTE 1, CTE 2, final select.
4. Write incrementally; check on 2-3 rows by hand.
5. Reconcile row counts after joins; mention indexes and performance.

SAY IT IN 20 SECONDS

- "Before joining, I check the grain of each table."
- "Top N per group: rank in a CTE, filter outside."
- "Retention: cohort by first day, divide by cohort size."
- "WHERE filters rows, HAVING filters groups."
- "I test on two or three rows by hand."

NUMPY ARRAYS

```
import numpy as np
a = np.array([[1, 2, 3], [4, 5, 6]]) # shape (2, 3)
np.zeros((2, 3)); np.ones(3); np.eye(3)
np.arange(0, 10, 2); np.linspace(0, 1, 5)
a.shape, a.ndim, a.dtype, a.size
a.reshape(3, 2); a.reshape(-1); a.T
a[0, 1]; a[:, 1]; a[a > 2] # index, column, mask
np.where(a > 2, 1, 0) # vectorised if/else
```

- One typed block of memory; operations are vectorised in C, avoid Python loops.
- Basic slices are **views** (edit the original); fancy and boolean indexing return **copies**. `a.copy()` to be safe.
- `np.random.default_rng(42).normal(size=5)`: always seed.
- `ravel()` is a view when possible, `flatten()` always a copy; check with `np.shares_memory(a, b)`.
- `a[:, None]` / `np.expand_dims` add an axis, `np.squeeze` drops length-1 axes. `np.stack` makes a new axis; `vstack/hstack/column_stack` reuse one. `np.split(a, 2)`.
- `np.argwhere(a > 2)`: one [row, col] per hit. `np.isin(a, [...])`.

AXIS AND AGGREGATION

- `axis=0` collapses rows (one result per column); `axis=1` collapses columns (one per row).
- `a.sum(axis=0)`, `.mean`, `.std(ddof=1)`, `.min/.max`, `.argmax`, `.cumsum`, `np.percentile(a, 90)`, `np.unique(a, return_counts=True)`.
- `keepdims=True` keeps the axis so the result broadcasts back.
- Sorting: `np.sort`, `np.argsort` (indices); `stack`: `np.concatenate`, `np.vstack`, `np.hstack`.
- Linear algebra: `a @ b` (`matmul`), `np.dot`, `np.linalg.inv`, `.solve`, `.eig`, `.norm`.
- Regression: `np.linalg.lstsq(X, y, rcond=None)[0]` with a ones column in `X`; safer than solving `X.T @ X`.

TRAP `a * b` is element-wise, `a @ b` is the matrix product: both run on square arrays. `np.std` divides by `n` (`ddof=0`); pandas `.std()` by `n - 1`. Pass `ddof` explicitly.

NUMPY EXTRAS

- Floats: `np.isclose(a, b)`, `np.nan` is a float; `np.nanmean`, `np.nansum`, `np.isnan(a)`; `np.inf`.
- `np.clip(a, lo, hi)`, `np.diff(a)`, `np.cumsum/cumprod`, `np.corrcoef(x, y)`, `np.histogram(a, bins=10)`, `np.searchsorted`.
- Dtypes: ints overflow silently (`int8`), `astype(float)`, `np.float32` halves memory. Mixed lists become `object`.
- Vectorise, then `np.vectorize` only as a convenience (it still loops). `np.select([c1, c2], [v1, v2], default)` for many branches.
- Random: `rng = np.random.default_rng(0)`; `rng.integers`, `rng.choice(a, size, replace=False)`, `rng.permutation`.

BROADCASTING RULES

Compare shapes from the **right**. Two dimensions are compatible if equal or one is 1 (the 1 is stretched). Missing leading dimensions count as 1.

Shapes	Result
(3, 4) with (4,)	(3, 4) subtract column means
(3, 4) with (3, 1)	(3, 4) subtract row means (use <code>keepdims</code>)
(3, 1) with (1, 4)	(3, 4) outer-style grid
(3, 4) with (3,)	error

Pairwise distances: `np.linalg.norm(x[:, None] - y[None], axis=2)`.

PANDAS: LOOK, SELECT, FILTER

```
import pandas as pd
df = pd.read_csv("f.csv", parse_dates=["d"])
df.shape; df.dtypes; df.head(); df.info()
df.describe(include="all"); df.isna().sum()
df["a"]; df[["a", "b"]] # Series / DataFrame
df.loc[rows, cols] # by label (end included)
df.iloc[0:3, 1:4] # by position (end excluded)
df[(df.a > 1) & (df.b == "x")] # & | ~ with brackets
df.query("a > 1 and b == 'x'")
df.loc[df.a > 1, "b"] = 0 # safe assignment
```

- Never chain `df[mask]["b"] = 0`: it may write to a copy (`SettingWithCopy`). Use `.loc`.
- `df.assign(c=df.a * 2)`, `df.rename(columns=...)`, `df.drop(columns=...)`, `df.astype({"a": "int32"})`.

GROUPBY, AGG, TRANSFORM

```
df.groupby("g")["x"].mean()
df.groupby("g").agg(n=("x", "size"),
                    avg=("x", "mean"))
df.groupby(["g", "h"], as_index=False).sum()
df["share"] = df.x / df.groupby("g")["x"].transform("sum")
df.groupby("g").head(2) # first rows per group
df.groupby("g")["x"].rank(ascending=False)
df.groupby("g")["x"].shift(1) # previous in group
df.groupby("g")["x"].cumsum()
```

- `agg` shrinks each group to one row; `transform` returns one value per original row (same index).
- `size` counts rows, `count` counts non-`NaN`, `nunique` distinct.

COMBINE AND RESHAPE

```
pd.merge(a, b, on="id", how="left")
a.merge(b, left_on="x", right_on="y")
pd.concat([a, b], axis=0, ignore_index=True)
a.join(b, on="k") # join on index
df.pivot_table(index="r", columns="c",
               values="v", aggfunc="sum")
df.melt(id_vars="id", var_name="k", value_name="v")
df.explode("tags"); df.stack(); df.unstack()
```

- `merge(..., validate="one_to_one")` catches key duplicates; `indicator=True` adds `_merge` (anti-join: `left_only`).
- `how`: `inner` (default), `left`, `right`, `outer`.

MISSING, DUPLICATES, TYPES

```
df.isna().mean() # share missing
df.dropna(subset=["a"]); df.dropna(how="all")
df["a"].fillna(df["a"].median())
df.fillna({"a": 0, "b": "NA"}); df.ffill()
df.duplicated(subset=["id"]).sum()
df.drop_duplicates("id", keep="last")
pd.to_datetime(df.d); pd.to_numeric(s, errors="coerce")
df["c"] = df["c"].astype("category")
```

- `NaN` $\neq$ `NaN`; use `isna()`. Mean skips `NaN` by default (`skipna=True`).
- Sort before `drop_duplicates(keep="last")` to keep the latest.
- Memory: `category` for low-cardinality strings, `downcast` numerics, read only needed columns (`usecols`), `chunks`.

STRINGS AND DATES

```
df.s.str.lower().str.strip().str.contains("x")
df.s.str.split(" ", expand=True); .str.extract(r"(\d+)")
df.d.dt.year; .dt.month; .dt.dayofweek; .dt.floor("D")
df.set_index("d").resample("W").sum()
df.x.rolling(7).mean(); df.x.expanding().max()
df.x.pct_change(); df.x.diff(); df.x.shift(7)
pd.cut(df.x, bins=[0, 18, 65, 120]) # fixed bins
pd.qcut(df.x, 4) # quantile bins
```

SQL -> PANDAS

SQL	pandas
WHERE	boolean mask / query
SELECT a, b	df[["a", "b"]]
GROUP BY ... AVG	groupby().agg()
HAVING	filter the grouped result
JOIN	merge(how=...)
ORDER BY ... LIMIT	sort_values().head()
COUNT(DISTINCT x)	nunique()
ROW_NUMBER	groupby().cumcount() + 1
RANK	groupby().rank(method="min")
LAG	groupby().shift(1)
SUM() OVER	groupby().transform("sum")
CASE WHEN	np.where / np.select
UNION ALL	pd.concat
DISTINCT	drop_duplicates()
anti-join	merge(indicator=True) then left_only

WORKED PATTERNS

```
# top 2 per group
df.sort_values("x", ascending=False).groupby("g").head(2)
# latest row per user
df.sort_values("ts").drop_duplicates("u", keep="last")
# conversion by group
df.groupby("g").agg(rate=("conv", "mean"))
# streak id: new id when the value changes
s = (df.v != df.v.shift()).cumsum()
# cohort retention table
df["m"] = df.d.dt.to_period("M")
first = df.groupby("u").m.transform("min")
df["age"] = (df.m - first).apply(lambda p: p.n)
df.pivot_table(index=first, columns="age",
                values="u", aggfunc="nunique")
```

State the output grain first; build in steps and print after each.

INDEX, APPLY, WINDOW HELPERS

- df.set_index("k"), reset_index(), sort_index(); label slicing needs a sorted index. df.loc["a":"c"] includes "c".
- Series.map({...}) / .map(func) elementwise; df.apply(func, axis=0/1) per column / row; applymap/DataFrame.map per cell.
- df.nlargest(3, "x"), df.sample(frac=.2, random_state=0), df.clip(lower, upper), df.where(cond, other).
- Correlation: df.corr(numeric_only=True) (Pearson), df.corr(method="spearman"), df.cov().
- Counts: s.value_counts(normalize=True), pd.crosstab(a, b), s.nunique().
- Split data: from sklearn.model_selection import train_test_split; train_test_split(X, y, test_size=.2, stratify=y, random_state=0).
- Display: pd.set_option("display.max_columns", 50); save with df.to_csv("o.csv", index=False), to_parquet.

CLEAN, ENCODE, RESHAPE: EXTRAS

```
pd.to_datetime(s, format="%Y-%m-%d", errors="coerce")
s.str.extract(r"([A-Z]+)-(\d+)") # groups → columns
s.str.contains("x", case=False, na=False)
s.dt.tz_localize("UTC").dt.tz_convert("Asia/Kolkata")
pd.get_dummies(df, columns=["c"], drop_first=True)
s.ffill(); s.bfill(); s.interpolate() # ordered data
df.dropna(axis=1, thresh=int(.5 * len(df)))
df.reindex(labels) # missing labels → NaN
df.query("a > @limit") # @ reads a variable
pd.read_sql(query, conn); df.to_sql("t", conn)
pd.crosstab(a, b, normalize="index", margins=True)
s.ewm(span=7).mean(); s.cummax(); s.shift(-1) # lead
df.pipe(func, arg) # own step in a chain
df.memory_usage(deep=True).sum()
```

- errors="coerce" turns bad values into NaN / NaT: count them afterwards. astype fails on the first bad value.
- pivot only reshapes (duplicate pairs raise); pivot_table aggregates; melt goes wide to long.
- Stacked series: groupby("g")["x"].shift(1), never a plain shift. A lead (shift(-1)) is a target, not a feature: leakage. bfill and interpolate also look ahead.
- Speed, fastest first: NumPy arrays, pandas vectorised, np.where/np.select, comprehension over zip(df.a, df.b), itertuples, apply(axis=1), iterrows (thousands of times slower).
- category pays when distinct values are far fewer than rows (a 200k-row table: 24.5 MB to 2.1 MB).

SPEED AND GOTCHAS

- Prefer vectorised column ops; apply(axis=1) is a Python loop (slow). np.where, .str, .dt, map(dict) are faster.
 - iterrows is slow; itertuples is faster; vectorise if you can.
 - inplace=True rarely saves memory; assign back instead.
 - = on a column of strings with NaN gives False for NaN. df.a.isin([...]) for membership.
 - df.copy() before modifying a slice you keep.
- TRAP** After a merge, compare row counts: duplicate keys fan out rows. groupby drops NaN keys by default (dropna=False to keep). Integer columns with NaN become float.

CHART CHOICE

Question	Chart
Compare categories	bar (start at zero)
Trend over time	line
Distribution of one variable	histogram, box, violin
Relationship of two numbers	scatter (+ trend line)
Many numeric pairs	correlation heatmap, pair plot
Part of a whole	stacked bar (pie only for 2-3 parts)
Ranking	sorted horizontal bar

```
import matplotlib.pyplot as plt, seaborn as sns
fig, ax = plt.subplots(figsize=(6, 3))
sns.histplot(df.x, bins=30, ax=ax)
sns.boxplot(data=df, x="g", y="v")
sns.heatmap(df.corr(numeric_only=True), annot=True)
ax.set(title="Takeaway as title", xlabel="x (unit)")

fig, axs = plt.subplots(1, 2, figsize=(8, 3))
sns.scatterplot(data=df, x="a", y="b", hue="g")
ax.set_yscale("log"); ax.tick_params(axis="x", rotation=45)
fig.tight_layout(); fig.savefig("f.png", dpi=150)
```

Label axes with units, one colour unless colour means something, avoid dual axes and 3-D, put the takeaway in the title, log scale for skewed data.

EDA ROUTINE

- Shape, dtypes, a few rows, memory.
- Missing values by column and by pattern: why missing?
- Duplicates and impossible values (negative age, future date).
- Each variable's distribution: skew, outliers, rare categories.
- Target versus each feature; correlations (leads, not proof).
- Segments and time trends; check data drift.
- Write findings and next steps. Decide on training data only.

SAY IT IN 20 SECONDS

- "transform keeps the rows, agg collapses them."
- "Broadcasting aligns shapes from the right; 1 stretches."
- "loc is label-based and inclusive, iloc positional and exclusive."
- "Slices are views, boolean masks are copies."

BIG-O QUICK REFERENCE

Class	Name	Typical example
O(1)	constant	dict lookup, array index
O(log n)	logarithmic	binary search, heap push/pop
O(n)	linear	one pass, hash-map solutions
O(n log n)	linearithmic	sorting, merge sort, heap of n
O(n²)	quadratic	nested loops, bubble sort
O(2^n), O(n!)	exponential, factorial	subsets, permutations

- n = 10^5 needs O(n log n) or better; n = 10^3 allows O(n²); n <= 20 allows O(2^n).
- Drop constants and lower-order terms; state **time and space** separately; recursion uses stack space O(depth).
- Amortised O(1): dynamic array append, hash-table insert.

DATA STRUCTURES: WHAT EACH IS GOOD AT

Structure	Access	Search	Insert/delete	Use for
array / list	O(1)	O(n)	O(n) middle, O(1) end	indexed data
hash map / set		O(1) avg	O(1) avg	counting, lookup, dedupe
linked list	O(n)	O(n)	O(1) at node	fast splice, queues
stack (list)	top O(1)		push/pop O(1)	undo, parsing, DFS
queue (deque)			O(1) both ends	BFS, windows
heap (heapq)	min O(1)		O(log n)	top-k, scheduling
BST balanced		O(log n)	O(log n)	ordered data
trie		O(len)	O(len)	prefix search
graph (adj. list)				networks, dependencies

- Sorting: Python sorted is Timsort, O(n log n), stable. Merge sort O(n log n) stable; quicksort O(n log n) average, O(n²) worst; counting sort O(n + k).

CLUE -> PATTERN

If the problem says	Think
pair / complement / seen before / count	hash map or set
sorted array, pair or triplet sum	two pointers
longest / shortest substring or subarray with a rule	sliding window
sum of a range, many queries	prefix sums
sorted, or "minimum value such that"	binary search (on answer)
matching brackets, next greater element	stack, monotonic stack
shortest path, levels, fewest steps	BFS
all paths, components, tree recursion	DFS
top k, k-th largest, merge k sorted	heap
overlapping intervals	sort by start, then merge
count ways, min cost, choices at each step	dynamic programming
all subsets, permutations, combinations	backtracking
connected groups, cycle in undirected graph	union-find

INTERVIEW ROUTINE

1. Restate; ask about input size, duplicates, negatives, empty input, sorted?
 2. Say a brute force and its cost; then improve (hash map? sort? pointers?).
 3. State the plan; code; **trace** on a small example.
 4. Edge cases: empty, one element, all same, duplicates, negatives, large n.
 5. State time and space. Target: easy in ~10 minutes, medium in ~20.
- Learn ~8 patterns well; re-solve problems from a blank page a week later.

PYTHON HELPERS FOR DSA

```
from collections import Counter, defaultdict, deque
import heapq, bisect, math
float("inf"); math.inf; sys.setrecursionlimit(10**6)
sorted(xs, key=lambda x: (-x[1], x[0]))
bisect.bisect_left(xs, x) # lower bound
divmod(a, b); a // b; a % b # floor division
```

heapq is a min-heap: push -x for a max-heap, tuples (priority, item) for ordering. deque.popleft() is O(1); list.pop(0) is O(n).

HASH MAP: TWO-SUM, COUNTING

```
def two_sum(nums, t):
    seen = {}
    for i, x in enumerate(nums):
        if t - x in seen:
            return [seen[t - x], i]
        seen[x] = i
# subarray sum == k (prefix sums + map)
cnt = Counter({0: 1}); s = res = 0
for x in nums:
    s += x
    res += cnt[s - k]
    cnt[s] += 1
```

TWO POINTERS (SORTED ARRAY)

```
l, r = 0, len(a) - 1
while l < r:
    s = a[l] + a[r]
    if s == t: return l, r
    if s < t: l += 1
    else: r -= 1
```

Also: remove duplicates in place (slow/fast pointer), reverse, merge two sorted lists, container with most water, 3-sum (sort, fix one, two pointers).

SLIDING WINDOW

```
# longest substring without repeats
last = {}; l = best = 0
for r, ch in enumerate(s):
    if ch in last and last[ch] >= l:
        l = last[ch] + 1
    last[ch] = r
    best = max(best, r - l + 1)
```

Fixed size k: add a[r], drop a[r - k]. Variable: expand right, shrink left while the rule is broken.

BINARY SEARCH

```
def bs(a, x):
    lo, hi = 0, len(a)
    while lo < hi:
        mid = (lo + hi) // 2
        if a[mid] < x: lo = mid + 1
        else: hi = mid
    return lo
```

On the answer: binary search `lo..hi` with `ok(mid)` monotonic (min capacity, min speed). Avoid `(lo + hi)` overflow in other languages.

STACK AND MONOTONIC STACK

```
def valid(s):
    pair = {"(": ")", "[": "]", "{": "}"
    st = []
    for c in s:
        if c in pair:
            if not st or st.pop() != pair[c]: return False
            else: st.append(c)
    return not st
# next greater element
res = [-1] * n; st = []
for i, x in enumerate(a):
    while st and a[st[-1]] < x: res[st.pop()] = x
    st.append(i)
```

BFS AND DFS (GRID / GRAPH)

```
def bfs(start, adj):
    seen = {start}; q = deque([(start, 0)])
    while q:
        node, d = q.popleft()
        for nb in adj[node]:
            if nb not in seen:
                seen.add(nb); q.append((nb, d + 1))
def dfs(node, adj, seen):
    seen.add(node)
    for nb in adj[node]:
        if nb not in seen: dfs(nb, adj, seen)
```

Grid neighbours: for dr, dc in `((1,0), (-1,0), (0,1), (0,-1))`, check bounds. BFS = shortest path in an unweighted graph; mark visited when enqueued. Cycle in a directed graph: DFS with colours or topological sort (Kahn: in-degrees).

HEAP: TOP-K, MERGE INTERVALS

```
heapq.nlargest(k, xs)
h = []
for x in xs:
    heapq.heappush(h, x)
    if len(h) > k: heapq.heappop(h)
# merge intervals
out = []
for s, e in sorted(iv):
    if out and s <= out[-1][1]:
        out[-1][1] = max(out[-1][1], e)
    else: out.append([s, e])
```

DYNAMIC PROGRAMMING

```
from functools import lru_cache
@lru_cache(None)
def fib(n): return n if n < 2 else fib(n-1) + fib(n-2)
# coin change: fewest coins
dp = [0] + [inf] * amt
for a in range(1, amt + 1):
    for c in coins:
        if c <= a: dp[a] = min(dp[a], dp[a - c] + 1)
```

Recipe: define the state, the recurrence, the base case, the order. Top-down (memoised recursion) or bottom-up table. Classics: climbing stairs, knapsack, longest increasing subsequence ($O(n \log n)$) with bisect), edit distance, longest common subsequence.

BACKTRACKING, LINKED LIST, TREE

```
def subsets(a):
    res = []
    def go(i, cur):
        if i == len(a): res.append(cur[:]); return
        go(i + 1, cur)
        cur.append(a[i]); go(i + 1, cur); cur.pop()
    go(0, []); return res
# reverse linked list
prev = None
while head:
    head.next, prev, head = prev, head, head.next
# tree height
def h(t): return 0 if not t else 1 + max(h(t.left), h(t.right))
```

Inorder of a BST is sorted. Level order = BFS with a queue.

GRAPH EXTRAS: DIJKSTRA, TOPOLOGICAL SORT, UNION-FIND

```
# Dijkstra (non-negative weights),  $O((V+E) \log V)$ 
dist = {src: 0}; h = [(0, src)]
while h:
    d, u = heapq.heappop(h)
    if d > dist.get(u, inf): continue
    for v, w in adj[u]:
        if d + w < dist.get(v, inf):
            dist[v] = d + w; heapq.heappush(h, (d + w, v))
# union-find
def find(x):
    while p[x] != x: p[x] = p[p[x]]; x = p[x]
    return x
def union(a, b): p[find(a)] = find(b)
```

Topological order (Kahn): queue nodes with in-degree 0, pop, decrement neighbours; if fewer than V nodes output, there is a cycle. Weighted graph with negative edges: Bellman-Ford.

BITS, MATH, STRINGS

- $x \& 1$ odd; $x >> 1$ half; $x \& (x - 1)$ clears lowest set bit; $x \wedge x == 0$ (single number); $1 \ll k$.
- Primes: sieve of Eratosthenes $O(n \log \log n)$. `math.gcd`, `pow(a, b, mod)`.
- Palindrome expand-around-centre $O(n^2)$; anagram by sorted key or Counter; strings are immutable, build with a list and `join`.
- Matrix: transpose `zip(*m)`, rotate 90 clockwise `[list(r)[::-1] for r in zip(*m)]`; prefix sums per row for 2-D range sums.

COMMON MISTAKES

TRAP Off-by-one in loops and binary search; infinite loop when `lo/hi` do not move. Mutating a list while iterating; appending `cur` instead of `cur[:]` in backtracking. Forgetting to mark visited before enqueueing (BFS) or the base case (recursion). Using `list.pop(0)` as a queue, `+=` on strings in a loop, or a default mutable argument.

- Test: empty input, one element, duplicates, negative numbers, a sorted and a reverse-sorted input.

SAY IT IN 20 SECONDS

- "Brute force is $O(n^2)$; a hash map makes it $O(n)$ time, $O(n)$ space."
- "Sliding window: the window is valid or I shrink it."
- "BFS finds the shortest path in an unweighted graph."
- "Dynamic programming = state, recurrence, base case."

DESCRIPTIVE STATISTICS

- **Mean** = sum/n (pulled by outliers). **Median** = middle value (robust). **Mode** = most frequent.
- **Variance** (sample) = $\sum(x - \text{mean})^2 / (n - 1)$; population divides by n. **SD** = $\sqrt{\text{variance}}$, same units as x.
- **IQR** = $Q3 - Q1$. Outlier rule: below $Q1 - 1.5 \cdot \text{IQR}$ or above $Q3 + 1.5 \cdot \text{IQR}$.
- **z-score** = $(x - \text{mean}) / \text{SD}$. Skew: right-skewed (income) means mean > median.
- **Covariance** = $\sum(x - \text{mx})(y - \text{my}) / (n - 1)$. **Correlation r** = $\text{cov}(\text{sx}, \text{sy})$, from -1 to 1: linear association only, units-free. $r = 0$ does not mean independent.
- Dividing by $n - 1$ (Bessel) corrects for estimating the mean from the same data.
- Robust spread: **MAD** = median $|x - \text{median}|$.

TRAP Correlation is not causation; confounders and reverse causation. Simpson's paradox: a trend reverses when groups are combined.

PROBABILITY RULES

- $0 \leq P(A) \leq 1$; $P(\text{not } A) = 1 - P(A)$. "At least one" = $1 - P(\text{none})$.
- Addition: $P(A \text{ or } B) = P(A) + P(B) - P(A \text{ and } B)$.
- Multiplication: $P(A \text{ and } B) = P(A) P(B | A)$. Independent: $P(A \text{ and } B) = P(A) P(B)$.
- Conditional: $P(A | B) = P(A \text{ and } B) / P(B)$.
- Total probability: $P(B) = \sum P(B | A_i) P(A_i)$.
- **Mutually exclusive** (cannot both happen) is not **independent**; exclusive events with $P > 0$ are dependent.
- Counting: $n!$ orderings; $nPr = n! / (n - r)!$; $nCr = n! / (r!(n - r)!)$.
- Expectation: $E[X + Y] = E[X] + E[Y]$ always (linearity). $\text{Var}(aX + b) = a^2 \text{Var}(X)$; $\text{Var}(X + Y) = \text{Var } X + \text{Var } Y$ if independent.

HANDY FACTS AND INEQUALITIES

- Markov: $P(X \geq a) \leq E[X] / a$ ($X \geq 0$). Chebyshev: $P(|X - \mu| \geq k\sigma) \leq 1/k^2$, any distribution (75% within 2σ).
- Percentiles: median = 50th, quartiles 25/75th; for the normal, the 97.5th percentile is $\mu + 1.96\sigma$.
- Skewness: sign gives the long tail; kurtosis: heavy tails. Log-transform right-skewed positives.
- Regression slope = $r(\text{sy}/\text{sx})$. Correlation is symmetric; regression is not.
- Law of total expectation: $E[X] = \sum E[X | A] P(A)$. Variance of a sum of n iid = $n\sigma^2$.
- Independence of events: $P(A | B) = P(A)$. Conditional independence differs from marginal independence.

- Expected value of a game = $\sum \text{value} \times \text{probability}$; decide with EV, mention variance.

BAYES' THEOREM

$P(H | E) = P(E | H) P(H) / P(E)$, posterior = likelihood $\times$ prior / evidence.

- Worked example: prevalence 1%, sensitivity 99%, false-positive 5%. $P(+)$ = $0.99 \cdot 0.01 + 0.05 \cdot 0.99 = 0.0594$. $P(\text{disease} | +)$ = $0.0099 / 0.0594 = 16.7\%$.
- Natural frequencies: of 10,000 people, 100 sick (99 test positive), 9,900 healthy (495 test positive): 99 of 594.
- Base-rate neglect: ignoring a small prior. Sensitivity = $TP / (TP + FN)$; specificity = $TN / (TN + FP)$.
- Odds form: posterior odds = prior odds $\times$ likelihood ratio.

CLASSIC PUZZLES

Puzzle	Answer
Two dice sum to 7	$6/36 = 1/6$
Two children, at least one girl, both girls	$1/3$
Birthday: 23 people share a birthday	> 50% (253 pairs)
Monty Hall	switch wins 2/3
Flips to first head	$E = 2$
Flips to HH / HT	$6 / 4$
Rolls to first six	6
Coupon collector, n types	$n(1 + 1/2 + \dots + 1/n)$

Method: define the sample space, count, then check by simulation.

DISTRIBUTIONS

Distribution	Mean	Variance	Use
Bernoulli(p)	p	$p(1 - p)$	one yes/no trial
Binomial(n, p)	np	$np(1 - p)$	successes in n trials
Geometric(p)	$1/p$	$(1 - p)/p^2$	trials to first success
Poisson(λ)	λ	λ	events per interval, rare and independent
Uniform(a, b)	$(a + b)/2$	$(b - a)^2/12$	equally likely
Normal(μ, σ^2)	μ	σ^2	sums and averages, errors
Exponential(λ)	$1/\lambda$	$1/\lambda^2$	waiting time, memoryless
Beta(α, β)	$\alpha/(\alpha + \beta)$		probability of a probability

- Binomial with large n, small p $\rightarrow$ Poisson(np). Normal approx when np and $n(1 - p) > 10$.
- Normal: 68% within 1σ , 95% within 2σ (1.96σ), 99.7% within 3σ . z crit: 1.645 (90%), 1.96 (95%), 2.576 (99%).
- t has heavier tails than normal, approaches it as df grows. Chi-square = sum of squared normals. F = ratio of variances.
- Log-normal / power law for incomes, latencies: report medians and percentiles.

PROBABILITY RECIPES

- Binomial: $P(X = k) = C(n, k) p^k (1 - p)^{n-k}$. Poisson: $P(X = k) = e^{-\lambda} \lambda^k / k!$.
- Draw without replacement (cards): multiply conditional probabilities, or use combinations: $P(2 \text{ aces}) = C(4, 2) / C(52, 2) = 1/221$.
- At least one success in n tries: $1 - (1 - p)^n$ (four rolls, a six: $1 - (5/6)^4 = 51.8\%$).
- Expected value of a die = 3.5; of a sum = sum of expectations. Gambler's game: $EV = \sum \text{payout} \times \text{probability}$.
- Conditional on a total: "given sum = 7, $P(\text{first die} = 3)$ " = $1/6$ by symmetry.
- Random variable transformations: $E[aX + b] = a E[X] + b$. Sum of independent normals is normal.
- Simulate when stuck: `np.random.binomial`, then count frequencies.

CLT AND LLN

- **LLN**: the sample mean converges to the true mean as n grows.
- **CLT**: the mean of n independent samples is approximately normal with mean μ and **standard error SE** = $\sigma / \sqrt{n}$, whatever the data's shape (finite variance), usually $n \geq 30$, more for heavy skew.
- $4 \times$ the data halves the error. It is about **means**, not about the data becoming normal.
- SE of a proportion = $\sqrt{p(1 - p)/n}$. SE of a difference = $\sqrt{SE_1^2 + SE_2^2}$.
- Law of rare events, bootstrap and the normal approximation all lean on the CLT.

CONFIDENCE INTERVALS

- Mean: sample mean $\pm t \cdot s/\sqrt{n}$ (z when n is large or σ known). Proportion: $p \pm 1.96 \cdot \sqrt{p(1-p)/n}$.
- **95% CI meaning:** if we repeated the sampling and the method many times, 95% of the intervals would cover the true value. Not "95% probability this interval holds it".
- Wider with more variance, higher confidence, smaller n. Margin of error = critical value $\times$ SE.
- Report the interval, not just the p-value: it shows effect size and uncertainty.

HYPOTHESIS TESTING

1. State H_0 (no effect) and H_1 .
 2. Choose alpha (0.05) and the test.
 3. Compute the statistic and **p-value**.
 4. Decide and interpret.
- **p-value** = P(data this extreme or more | H_0 true). Not P(H_0 true), not the effect size.
 - **Type I** (false positive, rate α): reject a true H_0 . **Type II** (miss, rate β): fail to reject a false H_0 . **Power** = $1 - \beta$, usually 80%.
 - Power rises with larger n, larger effect, lower variance, higher α . One-sided tests need a prior direction.
 - Statistical vs practical significance: huge n makes tiny effects significant. Report effect size (difference, Cohen's d, odds ratio).
 - Not significant $\neq$ no effect: low power gives wide intervals.

TRAP Peeking at results and stopping at the first $p < 0.05$ inflates false positives.

WHICH TEST?

Data and question	Test
Mean vs known value	one-sample t
Two independent means	Welch t-test (unequal variances)
Same units before/after	paired t-test
3+ group means	ANOVA, then post-hoc (Tukey)
Two proportions	two-proportion z-test / chi-square
Two categoricals	chi-square of independence; Fisher if counts < 5
Fit to a distribution	chi-square goodness of fit, KS
Two samples, non-normal / ordinal	Mann-Whitney U; paired: Wilcoxon
3+ groups, non-normal	Kruskal-Wallis
Linear association	Pearson r; rank-based: Spearman
Two distributions differ	two-sample KS

- t statistic = (sample mean - μ_0) / (s / $\sqrt{n}$). Two proportions: $z = (p_1 - p_2) / \sqrt{p(1-p)(1/n_1 + 1/n_2)}$ with pooled p.
- Assumptions: independence, roughly normal means (CLT), equal variances for classic t/ANOVA. Heavy tails: bootstrap or transform.

SAMPLE SIZE (PER GROUP)

- Means: $n \sim 16\sigma^2/\delta^2$ (80% power, $\alpha = 0.05$). Proportions: $n \sim 16p(1-p)/\delta^2$.
- Example: baseline 10%, detect +1pp: $16 \cdot 0.09/0.0001 \sim 14,400$ per group.
- Halving the detectable effect quadruples n. Duration = total n / daily eligible traffic, in whole weeks.

MULTIPLE TESTING

- m tests at $\alpha = 0.05$: P(at least one false positive) = $1 - 0.95^m$ (20 tests: 64%).
- **Bonferroni:** use α/m (controls family-wise error; strict). **Benjamini-Hochberg:** controls the false discovery rate; rank p-values, keep $p(i) \leq (i/m) \cdot q$.
- Pre-register one primary metric; treat post-hoc segment hits as hypotheses.

```
from scipy import stats
stats.ttest_ind(a, b, equal_var=False)
stats.ttest_rel(a, b)
stats.chi2_contingency(table)
stats.mannwhitneyu(a, b);
stats.ks_2samp(a, b)
stats.pearsonr(x, y);
stats.norm.ppf(0.975)
import statsmodels.stats.multitest as mt
mt.multipletests(p, method="fdr_bh")
from statsmodels.stats import power
power.TTestIndPower().solve_power(
    effect_size=0.2, power=0.8,
    alpha=0.05)
```

EFFECT SIZES AND ASSUMPTIONS

Quantity	Formula / use
Absolute lift	$p_{\text{treat}} - p_{\text{control}}$
Relative lift	$(p_{\text{treat}} - p_{\text{control}}) / p_{\text{control}}$
Cohen's d	difference in means / pooled SD (0.2 small, 0.5 medium, 0.8 large)
Odds ratio	$[p/(1-p)]_{\text{treat}} \div \text{control}$
R^2	$1 - \text{SS}_{\text{res}} / \text{SS}_{\text{total}}$

- Check normality with a histogram or Q-Q plot, equal variances with Levene's test, independence from the design (clustered users break it).
- Violations: transform (log), use Welch, use non-parametric tests, or bootstrap.
- Paired data: analyse differences; clustered data: aggregate to the randomisation unit.

OTHER ESSENTIALS

- **Sampling:** simple random, stratified (represent subgroups), cluster. Biases: selection, survivorship, non-response.
- **Bootstrap:** resample with replacement, recompute the statistic, take percentiles for a CI. **Permutation test:** shuffle labels.
- **MLE:** choose parameters maximising the likelihood; for a normal it gives the sample mean and variance (divides by n). **MAP** adds a prior.
- **Bayesian vs frequentist:** posterior = prior $\times$ likelihood gives probability statements about parameters; credible interval is a probability statement, a confidence interval is a procedure property.
- **Linear algebra:** eigenvector keeps direction under a matrix; PCA = eigenvectors of the covariance matrix; SVD = $U \Sigma V^T$.
- **Calculus:** the gradient points uphill; gradient descent steps against it; chain rule powers backprop.
- Regression: R^2 = explained variance share; coefficient t-test = estimate/SE.

SAY IT IN 20 SECONDS

- **p-value:** "how surprising this data is if nothing is going on".
- **CI:** "a range from a procedure that covers the truth 95% of the time".
- **CLT:** "averages are roughly normal with SD $\sigma/\sqrt{n}$ ".
- **Power:** "the chance to detect a real effect of this size".
- **Bayes:** "update the prior with the likelihood; never forget the base rate".
- Definition, tiny example, one caution.

EXPERIMENT LIFECYCLE

1. **Hypothesis and decision:** what will we do differently if it wins?
 2. **Metrics:** one primary (the decision metric), a few secondary, **guardrails** that must not worsen (latency, refunds, unsubscribes).
 3. **Unit of randomisation:** usually the user; analyse at the same unit. Exclusions defined up front.
 4. **Sample size, duration** from baseline, minimum detectable effect (MDE), α and power.
 5. **Validate setup:** A/A test, sample ratio mismatch (SRM), balance on pre-period metrics.
 6. **Run** the planned duration, whole weeks, no peeking decisions.
 7. **Analyse** with the pre-registered test; check guardrails and segments (as hypotheses).
 8. **Decide, document, monitor** after launch (holdout, long-term effect).
- Write the decision rule beforehand: ship if lift $\geq$ MDE and guardrails flat; iterate if direction good but inconclusive; stop otherwise.

METRIC TYPES

Type	Example	Note
Primary / north star	conversion, revenue per user	one, sensitive, tied to decision
Driver / diagnostic	click-through, add-to-cart	explain the primary
Guardrail	latency, crash rate, refund rate, retention	must not degrade
Ratio	CTR = clicks/impressions	unit mismatch: use per-user or delta method

- Good metrics: measurable, sensitive, hard to game, aligned with long-term value. Optimising one alone gets gamed (clicks up, satisfaction down).

- Per-user metrics with heavy tails (revenue): winsorise, log-transform, or use a bootstrap.

SAMPLE SIZE (PER GROUP)

- Two proportions: $n = \frac{(z\alpha/2 \cdot \sqrt{p(1-p)} + z\beta \cdot \sqrt{p_1(1-p_1) + p_2(1-p_2)})^2}{\delta^2}$, with p the pooled rate and $q = 1 - p$. Rule of thumb: $n \sim 16 \cdot p(1-p) / \delta^2$ ($\alpha = 0.05$, power 80%).
- Means: $n \sim 16\sigma^2 / \delta^2$ where δ is the absolute difference.
- z values: $\alpha = 0.05$ two-sided 1.96; power 80% 0.84; power 90% 1.28.

Baseline	+5% rel	+10% rel	+20% rel
2%	315k	81k	21k
5%	122k	31k	8.2k
10%	58k	15k	3.8k
20%	26k	6.5k	1.7k
50%	6.3k	1.6k	390

- Halving the MDE needs 4x the users. Duration = $(2n) / (\text{daily eligible users})$, rounded up to full weeks.
- Variance too high? Use CUPED, a more sensitive metric, a larger MDE, or longer duration.

RANDOMISATION AND EXPOSURE

- Assign by hashing the user id with an experiment salt: stable across visits, independent across experiments. Never assign by session if the metric is per user.
- **Ramp** 1% $\rightarrow$ 10% $\rightarrow$ 50% to catch bugs; the analysis uses the full-allocation period only.
- Log **exposure** (when a user actually sees the change) and analyse the triggered population to cut noise; keep the intent-to-treat view too.
- **Layers / mutually exclusive** experiments avoid interaction between concurrent tests. Keep a **global holdout** for long-term effects.
- Balance check: pre-experiment metrics and device mix should be equal across arms.

- Hypotheses: H0 no difference; H1 two-sided unless a one-sided direction is justified in advance.

ANALYSIS STEPS

1. Check SRM first: chi-square of observed vs expected split. 257.5k/242.5k out of 500k, expected 50/50: chi-square = 450, p tiny: a bug, do not trust the results.
2. Compute lift with a **confidence interval**, not just p.
3. Two proportions: $z = \frac{(p_2 - p_1) / \sqrt{p(1-p)(1/n_1 + 1/n_2)}}{}$, p pooled. Example: 10% vs 11%, 14,400 each: $z = 2.77$, $p = 0.006$.
4. Means: Welch t-test; heavy tails: bootstrap CI.
5. Guardrails and secondary metrics: expect noise; correct for many metrics.
6. Segments (platform, new vs returning) only as hypotheses, or pre-planned.
7. Practical significance: is the CI above the minimum worth shipping?

```
import statsmodels.stats.proportion as sp
z, p = sp.proportions_ztest([c2, c1], [n2, n1])
es = sp.proportion_effectsize(0.11, 0.10)
from statsmodels.stats import power
power.NormalIndPower().solve_power(es, power=0.8, alpha=0.05)
```

VARIANCE REDUCTION AND RATIO METRICS

- **CUPED:** $Y_{\text{adj}} = Y - \theta(X - \text{mean } X)$, $\theta = \text{cov}(X, Y) / \text{var}(X)$, X = pre-experiment metric. Variance shrinks by $(1 - r^2)$: $r = 0.7$ halves it.
- **Stratification / blocking** by platform or country; **paired designs** where possible.
- **Ratio metrics** (CTR, revenue per session): analyse per user or use the delta method; the unit of analysis must match the unit of randomisation.
- Winsorise at the 99th percentile for revenue; report both.

PITFALLS AND FIXES

Pitfall	Fix
Peeking and stopping at first $p < 0.05$	fixed horizon or sequential testing
Multiple metrics / segments	one primary, correct (BH), hypotheses
SRM (split off)	find the bug (bots, redirects, logging) and rerun
Novelty / primacy effect	run 2+ weeks, look at later cohorts
Underpowered test	compute n ; report CI, not "no effect"
Interference (marketplaces, social)	cluster, geo or switchback designs
Wrong unit (session vs user)	analyse at the randomisation unit
Seasonality, launches	full weeks, avoid holidays, run concurrently
Survivorship (analyse only active users)	intent-to-treat analysis
Simpson's paradox	stratify, check allocation by segment

- Weekday effects: run whole weeks. Carry-over: do not reuse exposed users.
- A/A tests should reject ~5% of the time at $\alpha = 0.05$.

ADVANCED DESIGNS

- **Sequential testing** (alpha spending, mSPRT): valid early looks. **Bayesian A/B**: $P(B > A)$ and expected loss; choose a prior; still need a stopping plan.
- **Multi-armed bandits** (epsilon-greedy, Thompson sampling): shift traffic to winners, less regret, weaker inference. Good for short-lived choices (headlines).
- **Cluster randomisation**: randomise groups (cities, classrooms); effective sample size is smaller (design effect = $1 + (m - 1) \cdot ICC$).
- **Switchback**: alternate treatment by time block in one market (pricing, dispatch).
- **Quasi-experiments** when you cannot randomise: **difference-in-differences** (treated change – control change, needs parallel trends), **synthetic control**, regression discontinuity, matching / propensity scores, instrumental variables.

- **Interleaving** for ranking and search; **holdouts** for long-term effects; **multivariate** tests need much more traffic.
- **Uplift modelling**: who changes behaviour because of treatment, not who is likely to convert anyway.
- **Network effects**: randomise by graph cluster; expect bias if you do not.

DECISION MATRIX

Result	Action
CI above MDE, guardrails flat	ship (maybe ramp 10 → 50 → 100%)
CI includes 0, adequate power	no detectable effect: stop or iterate
CI includes 0, underpowered	extend or redesign, do not call it flat
Primary up, guardrail down	trade-off: quantify, product call
Segment-only win	follow-up test, do not ship on it
SRM or tracking bug	invalid: fix and rerun

- Always say: effect size, interval, cost of being wrong, next step.

CASE-ANSWER SKELETON

1. Clarify the goal and the decision.
 2. Primary metric, guardrails, unit of randomisation, exclusions.
 3. Sample size, duration, ramp plan.
 4. Risks: novelty, interference, seasonality, instrumentation, SRM.
 5. Analysis plan: test, CI, segments, corrections.
 6. Decision rule for each outcome; follow-up and monitoring.
- **Cannot A/B test?** Staged rollout with holdout, difference-in-differences, synthetic control, or before-and-after with caveats.
 - **Two-sided marketplace pricing**: randomise by market or time; track both sides (rider wait, driver earnings, cancellations).
 - **Only 5,000 users a week, heavy-tailed revenue**: CUPED, winsorise, a proxy metric, larger MDE, longer run.
 - **+0.3% conversion but AOV down**: judge revenue per user with its CI.

PRE-LAUNCH AND REVIEW CHECKLIST

- Hypothesis, decision rule and primary metric written down. Sample size, duration, MDE agreed.

- Instrumentation tested: events fire once, in both arms, with the experiment id.
- Eligibility and exclusions (bots, employees, repeat users) defined; exposure logged.
- Guardrails and alerts (auto-stop on crashes or revenue collapse).
- After: SRM, novelty check (effect by week), CI, segments flagged as exploratory, write-up with the decision and learning.
- Tell stakeholders: "We estimate +X% (95% CI a to b). Ship because ... Risk ...".

WHERE TESTS GO WRONG IN PRACTICE

- Tracking changes in one arm only; caching, redirects, or app versions that never see the variant.
- Treatment that changes who is counted in the metric (more users reach checkout, conversion falls): use a denominator that is unaffected.
- Mixed-up control (existing users) vs treatment (new users): not a randomised comparison.
- Effects that decay (promotions), or build slowly (habit features): use cohorts and a holdout.
- Testing many small variants at once: low power and multiple-testing risk.

QUICK NUMBERS

- 95% CI half-width for a proportion $\sim 2 \cdot \sqrt{p(1-p)/n}$: $n = 10,000$, $p = 10\%$: $\pm 0.6pp$.
- Detect 1pp on a 10% base needs ~14,400 per group; 0.5pp needs ~57,800.
- 20 metrics at $\alpha = 0.05$: 64% chance of at least one false positive.
- Dates: avoid holidays, launches; at least one full business cycle.

SAY IT IN 20 SECONDS

- "Randomise at the user, size for the effect that matters, run full weeks, no peeking."
- "A significant lift with a falling guardrail is not a win."
- "SRM means stop and debug; do not interpret."
- "No significance with low power is not 'no effect'."

CORE CONCEPTS

- **Supervised:** labels. Regression (number) / classification (category). **Unsupervised:** structure without labels (clustering, dimensionality reduction). **Reinforcement:** reward from actions.
- Goal: **generalise** to unseen data. Split: train (fit), validation (tune), test (touch once for the final score).
- **Bias** = error from too simple a model (underfit). **Variance** = sensitivity to the training sample (overfit). Total error = $\text{bias}^2 + \text{variance} + \text{irreducible noise}$.

Symptom	Diagnosis	Fix
train bad, val bad	high bias	richer model, features, less regularisation
train good, val bad	high variance	more data, regularise, simpler, bagging, early stop
both good, prod bad	leakage / drift / skew	audit features, monitor

- **Learning curve:** error vs training size. Curves that converge high: bias. A persistent gap: variance (more data helps).
- Parametric (fixed number of weights: linear, logistic) vs non-parametric (grows with data: KNN, trees).
- Loss: MSE (regression, outlier-sensitive), MAE (robust), Huber, log loss / cross-entropy (classification), hinge (SVM).

REGULARISATION

- Add a penalty on weights: **L2 (ridge)** $\lambda \sum w^2$ shrinks smoothly; **L1 (lasso)** $\lambda \sum |w|$ sets some weights exactly to 0 (feature selection); **elastic net** mixes both (handles correlated features).
- Why L1 is sparse: the diamond-shaped constraint has corners on the axes; L1's pull is constant, L2's pull fades as $w \rightarrow 0$.
- Larger λ = simpler model (more bias, less variance). Choose λ by cross-validation. **Scale features first.**
- Also: dropout, early stopping, max depth, pruning, data augmentation, more data.

DATA LEAKAGE

Model sees information at training that will not exist at prediction time. Scores look great, production fails.

- **Target leakage:** a feature derived from the outcome ("refund issued" predicting fraud; "days since account closed" predicting churn).
- **Train-test contamination:** scaling, imputing, encoding or feature selection fitted on all data; duplicates across the split; SMOTE before splitting.
- **Temporal:** random split on time-ordered data (use forward chaining); features computed with future data.
- **Group:** same customer or patient in train and test (use group k-fold).
- Fix: split first, fit every step in a Pipeline on training folds only; ask "will this exist at prediction time?"; distrust near-perfect scores.

LINEAR REGRESSION

- $y = Xw + \epsilon$, fit by least squares: $w = (X^T X)^{-1} X^T y$ (normal equation) or gradient descent. Coefficient = change in y per unit of x , other features fixed.
- Assumptions (**LINE**): Linear relationship, Independent errors, Normal errors (for inference), Equal variance (homoscedasticity). Check residual plots, Q-Q plot.
- $R^2 = 1 - \text{SSres}/\text{SStot}$ (rises with every feature; use adjusted R^2 or validation error).
- **Multicollinearity:** correlated features give unstable coefficients; check VIF > 5-10; drop, combine, or use ridge.
- Outliers and leverage points pull the line; extrapolation is unsafe. Heteroscedasticity: log target, weighted least squares, robust SEs.

LOGISTIC REGRESSION

- $p = 1/(1 + e^{-z})$, $z = w \cdot x + b$. It models **log-odds** linearly: $\log(p/(1-p)) = z$. $\exp(w) = \text{odds ratio}$ per unit.
- Loss: log loss = $-[y \log p + (1-y) \log(1-p)]$, trained by maximum likelihood. Linear decision boundary. Multiclass: softmax.
- 0.5 is a default threshold, not a law: choose from the cost of errors (precision-recall curve).
- Regularised by default in sklearn ($C = 1/\lambda$). Probabilities are usually well calibrated; scale features for faster convergence.
- Good baseline: interpretable, fast, stable; weak on interactions unless you add features.

DECISION TREES

- Greedy recursive splits: choose the feature and threshold with the largest impurity reduction. **Gini** = $1 - \sum p^2$; **entropy** = $-\sum p \log_2 p$; regression uses variance.
- Example: node 50/50 has Gini 0.5; a split into 45/5 and 5/45 gives 0.18 each, gain 0.32.
- Controls: max_depth, min_samples_leaf, ccp_alpha (pruning). Deep trees overfit; small data changes alter the tree.
- No scaling needed, handles mixed types, captures interactions; step-like, poor at extrapolation.

RANDOM FOREST (BAGGING)

- Many trees on **bootstrap samples**, each split on a random feature subset ($\text{max_features} \sim \sqrt{p}$); average (regression) or vote. Variance falls; bias unchanged.
- **OOB score:** each tree is tested on rows it did not see (~37%): a free validation estimate.
- Importance: impurity-based favours high-cardinality features; prefer **permutation importance** or SHAP.
- Few hyperparameters ($n_estimators$, max_depth, min_samples_leaf); robust; cannot extrapolate beyond training range; slow, large models.

GRADIENT BOOSTING

- Trees built in **sequence**: each fits the residuals (negative gradient of the loss) of the ensemble so far; prediction = $\sum \text{learning_rate} \times \text{tree}$. Reduces **bias**; shallow trees (depth 3-8).
- Key knobs: learning_rate (small + many trees), n_estimators, max_depth, subsample, colsample_bytree, min_child_weight, L1/L2 (reg_alpha/lambda). Use **early stopping** on a validation set.
- **XGBoost:** regularised, second-order gradients, handles missing values, parallel. **LightGBM:** histogram bins, leaf-wise growth, fast on big data. **CatBoost:** native categorical handling, ordered boosting (less leakage).
- Usually the strongest model on tabular data. Overfits noisy labels; needs tuning; less interpretable (use SHAP).

ALGORITHM CARDS

Model	Idea	Scale?	Watch out
SVM	maximum-margin hyperplane; support vectors only; kernels (RBF) for curves	yes	C large = overfit, gamma large = wiggly; slow on big n
KNN	vote of k nearest points; no training	yes	slow predict, curse of dimensionality; choose k by CV
Naive Bayes	P(class) × product of P(feature given class) assuming conditional independence	no	poor probabilities; Laplace smoothing; great text baseline
Linear / logistic	weighted sum	yes	non-linearity needs features
Tree / forest / boost	thresholds	no	trees extrapolate poorly

- SVM margin = $2/|w|$; soft margin trades errors for width. KNN example: unscaled income (dollars) swamps age in distance, so scale first.
- Naive Bayes example: $P(\text{spam} \mid \text{"free"}) = 0.4 \times 0.5 / (0.4 \times 0.5 + 0.6 \times 0.05) = 87\%$.

ENSEMBLING

- Bagging**: parallel models on bootstrap samples, average: cuts variance (random forest).
- Boosting**: sequential error correction: cuts bias (GBM, AdaBoost).
- Stacking**: a meta-model learns from base models' **out-of-fold** predictions (in-sample predictions leak).
- Voting/averaging**: combine diverse models. Gains need diversity; costs latency and explainability.
- Pick an algorithm**: start with a simple baseline (logistic/linear) → gradient boosting for tabular → linear when explanation or regulation matters → KNN/SVM for small data → deep learning for images, text, audio.

OPTIMISATION

- Gradient descent: $w \leftarrow w - \eta \text{ grad } L(w)$. Too large η diverges (NaN loss); too small is slow. Scale features.

- Batch** (all data), **stochastic** (one row, noisy), **mini-batch** (32-512, the default). **Momentum** smooths steps; **Adam** keeps per-parameter adaptive rates (default for deep nets); **RMSprop**, **AdaGrad**.
- Convex losses (linear, logistic) have one global minimum; neural nets have many local ones and saddle points. Use learning-rate schedules and early stopping.
- Normal equation $O(p^3)$ vs GD $O(n \cdot p)$ per step: GD for many features.

CROSS-VALIDATION AND TUNING

- k-fold** (k = 5): train on k - 1 folds, validate on one, average. **Stratified** for classes, **group k-fold** for entities, **TimeSeriesSplit** (forward chaining) for time.
- Grid search** (all combinations), **random search** (more efficient), **Bayesian/Optuna** (learns from trials). Tune on validation folds, test set once. **Nested CV** for an honest estimate when tuning heavily.

```
from sklearn.pipeline import
make_pipeline
from sklearn.preprocessing import
StandardScaler
from sklearn.linear_model import
LogisticRegression as LR
from sklearn.model_selection import
GridSearchCV
pipe = make_pipeline(StandardScaler(),
LR())
gs = GridSearchCV(pipe,
{"logisticregression__C": [.1, 1,
10]}),
cv=5, scoring="roc_auc")
gs.fit(X_train, y_train)
gs.best_params_
```

Scaler inside the pipeline = refit per fold = no leakage.

TYPICAL HYPERPARAMETERS (SKLEARN)

Model	Tune first	Typical range
Ridge / Lasso	alpha	1e-4 to 1e2, log scale
LogisticRegression	C (1/λ), penalty	1e-3 to 1e2
RandomForest	n_estimators, max_depth, min_samples_leaf, max_features	200-500, 5-30, 1-20, sqrt
GradientBoosting / XGB	learning_rate, n_estimators, max_depth, subsample	.01-.3, 100-2000, 3-8, 1/2
SVM (RBF)	C, gamma	1e-2 to 1e3, 1e-4 to 1
KNN	n_neighbors, weights	3-30, uniform/distance
DecisionTree	max_depth, min_samples_leaf, ccp_alpha	3-15

INTERPRETING MODELS

- Linear/logistic**: coefficients (scaled features) and odds ratios; check significance with statsmodels.
- Trees/forests**: impurity importance (biased); **permutation importance** (shuffle a feature, measure the score drop) is better.
- SHAP**: per-prediction additive contributions; global bar and beeswarm plots. **PDP/ICE**: average effect of a feature. **LIME**: local linear surrogate.
- Calibration: reliability curve; fix with Platt scaling or isotonic regression (CalibratedClassifierCV). Needed when probabilities drive decisions.
- Correlated features share importance; interpretation is not causation.

EVERYDAY SKLEARN

```
from sklearn.ensemble import (
RandomForestClassifier as RF)
m = RF(n_estimators=300, random_state=0)
m.fit(X_tr, y_tr)
p = m.predict_proba(X_te)[: , 1]
from sklearn.model_selection import (
cross_val_score)
cross_val_score(m, X, y, cv=5,
scoring="f1")
from sklearn.metrics import roc_auc_score
roc_auc_score(y_te, p)
```

fit on train only; transform test with train-fitted objects; set random_state.

WHEN IT BREAKS: SYMPTOM -> TRY

Symptom	Try
Overfits	regularise, prune, more data, bagging, early stop
Underfits	features, interactions, bigger model, lower λ
Slow training	subsample, LightGBM, simpler model, fewer features
Unbalanced	class weights, threshold, PR-AUC (see Module 8)
log Unstable coefficients	drop collinear features, ridge
Offline great, online poor	leakage, train-serving skew, drift, wrong metric

SAY IT IN 20 SECONDS

- Bias-variance**: simple models underfit, flexible ones memorise noise.
- Bagging vs boosting**: average independent models vs fix errors in sequence.
- L1 vs L2**: L1 zeroes weights, L2 shrinks them.
- Leakage**: using information from the future or the label.
- Idea, reason, example, trade-off.

MISSING VALUES

- First ask **why**: MCAR (random), MAR (depends on other variables), MNAR (depends on the missing value itself, e.g. high earners skip income).

Option	When
drop rows / columns	few missing, MCAR; column > ~60% empty
median / mode impute	simple, robust baseline
group-wise impute	missingness differs by segment
KNN / iterative (MICE)	MAR with strong correlates
add a <code>was_missing</code> flag	missingness itself is signal
tree models (XGB/LightGBM)	handle NaN natively

- Fit imputers on **training data only** (inside the pipeline).

OUTLIERS

- Detect: IQR fences (Q1 – 1.5·IQR, Q3 + 1.5·IQR), z-score > 3 (inflated by the outlier itself), robust z with median and MAD, Isolation Forest.
- Decide: error (fix/drop), real but extreme (keep). Treat: winsorise (cap at 1st/99th percentile), log-transform, robust scaler, robust models (trees, Huber).
- Never delete points only because they hurt the model.

ENCODING CATEGORICALS

Method	Use	Caution
one-hot	few levels, linear/NN	wide with many levels; drop one for linear
ordinal	natural order (S < M < L)	do not order unordered labels
frequency / count	high cardinality	collisions of equal counts
target / mean encoding	high cardinality	leaks: compute out-of-fold, smooth
hashing	huge, streaming	collisions
embeddings	very high cardinality, NN	needs data
native (CatBoost, LightGBM)	boosting	built-in handling

- Unseen categories at serving: `handle_unknown="ignore"`, an "other" bucket.

SCALING AND TRANSFORMS

Scaler	Formula	Use
StandardScaler	$(x - \text{mean})/\text{SD}$	linear/logistic, SVM, KNN, PCA, NN
MinMaxScaler	$(x - \text{min})/(\text{max} - \text{min})$	bounded inputs, NN, images
RobustScaler	$(x - \text{median})/\text{IQR}$	outliers

- Trees, forests and boosting **do not need scaling**. Fit the scaler on train only.
- Log / log1p** for right-skewed positives (income, counts); Box-Cox / Yeo-Johnson for general skew; binning for non-linear thresholds (KBinsDiscretizer).

FEATURE IDEAS

- Dates**: hour, day of week, month, `is_weekend`, holiday, days since event; cyclic encoding ($\sin/\cos$ of $2\pi \cdot h/24$).
- Aggregates**: user mean/sum/count over 7/30/90 days, recency-frequency-monetary (RFM), ratios (spend per visit), trends (7-day vs 30-day).
- Interactions** (x1·x2), polynomials, domain features (BMI, distance), text length, TF-IDF, embeddings, geo distance.
- Window features** must use data **before** the prediction time only.
- Selection: **filter** (correlation, mutual information, chi-square, variance threshold), **wrapper** (recursive feature elimination), **embedded** (L1, tree importance, permutation importance). Remove constant, duplicate and leaky features; check stability over time.

FEATURES BY DATA TYPE

- Text**: length, counts, TF-IDF n-grams, embeddings; lower-case, strip, handle rare words.
- Time series**: lags, rolling mean/std, differences, expanding stats, calendar and holiday flags.
- Geo**: distance to points, clusters, region; **images**: pretrained embeddings.
- Counts/rates**: per-user averages, ratios with smoothing (add a prior) for small denominators.

- Target (mean) encoding recipe**: out-of-fold category means, smoothed: $(n \cdot \text{cat_mean} + m \cdot \text{global_mean})/(n + m)$.

DIMENSIONALITY REDUCTION

- PCA**: centre and **scale**, take eigenvectors of the covariance matrix; PC1 has maximal variance; components are orthogonal linear mixes. Keep enough for 90-95% explained variance (`explained_variance_ratio_`, scree plot). Unsupervised; loses interpretability; helps noise, speed, multicollinearity.
- t-SNE / UMAP**: non-linear embeddings for 2-D visualisation of clusters; distances between clusters and cluster sizes are **not** meaningful; not for feeding models (UMAP is better for that).
- LDA**: supervised, finds axes that best separate classes (at most classes – 1). **Autoencoders**: non-linear compression. **SVD**: matrix factorisation behind PCA and recommenders.

```
from sklearn.compose import
ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import (
    OneHotEncoder, StandardScaler)
from sklearn.pipeline import
make_pipeline
pre = ColumnTransformer([
    ("num", make_pipeline(
        SimpleImputer(strategy="median"),
        StandardScaler()), num_cols),
    ("cat", OneHotEncoder(
        handle_unknown="ignore"), cat_cols)])
```

CONFUSION MATRIX AND CLASSIFICATION METRICS

	Predicted +	Predicted -
Actual +	TP	FN
Actual -	FP	TN

Metric	Formula	Question answered
Accuracy	$(TP + TN) / \text{all}$	share correct (misleads when imbalanced)
Precision	$TP / (TP + FP)$	of flagged, how many are real
Recall (TPR, sensitivity)	$TP / (TP + FN)$	of real, how many found
Specificity	$TN / (TN + FP)$	of negatives, how many cleared
FPR	$FP / (FP + TN)$	false alarm rate
F1	$2PR / (P + R)$	harmonic mean; F β weights recall $\beta \times$ more
Balanced accuracy	$(TPR + TNR) / 2$	imbalance-aware accuracy
MCC	all four cells	single robust score

- Example: 100 flagged, 60 real, 120 real in total: precision 60%, recall 50%, F1 0.545.
- A model that always says "no fraud" on 99:1 data: accuracy 99%, recall 0%.
- Raise the threshold $\rightarrow$ precision up, recall down; lower it $\rightarrow$ the reverse.

WORKED EXAMPLE

1,000 cases, 50 fraud. Model flags 80: 40 are fraud (TP = 40), so FP = 40, FN = 10, TN = 910.

- Accuracy = $950 / 1000 = 95\%$. Precision = $40 / 80 = 50\%$. Recall = $40 / 50 = 80\%$. F1 = 0.615. FPR = $40 / 950 = 4.2\%$.
- A "never fraud" model: accuracy 95%, recall 0%. Same accuracy, useless.
- Cost view: FN costs \$500, FP costs \$5: cost = $10 \cdot 500 + 40 \cdot 5 = \$5,200$. Lowering the threshold trades FNs for cheap FPs until cost stops falling.

CURVES AND PROBABILITIES

- **ROC-AUC:** P(random positive ranks above random negative); 0.5 = chance, 1 = perfect. Threshold-free, ranking quality; looks optimistic with rare positives.
- **PR-AUC / average precision:** focuses on the positive class; baseline = prevalence. Prefer for imbalance.

- **Log loss** (cross-entropy) punishes confident wrong probabilities; **Brier score** = mean squared probability error. **Calibration:** of cases scored 0.7, ~70% are positive.
- **KS statistic** (max gap between cumulative good/bad), **Gini** = $2 \cdot \text{AUC} - 1$ (credit scoring), **lift/gain** at top decile.

CHOOSING THE METRIC

Situation	Metric
Missing a case is costly (cancer, fraud)	recall at acceptable precision
False alarms are costly (spam block, ad fatigue)	precision
Both matter	F1 or the PR curve
Known costs	expected cost = $c_{\text{FN}} \cdot \text{FN} + c_{\text{FP}} \cdot \text{FP}$, pick the threshold that minimises it
Limited review capacity (50 cases/day)	precision@K
Balanced, symmetric costs	accuracy / ROC-AUC
Probabilities are used (pricing, risk)	log loss, calibration

- Say what the metric means for the business, then **tune the threshold**, not only the model.

REGRESSION, RANKING, CLUSTERING

- **MAE** mean |error| (robust, in units). **MSE** mean squared (punishes big misses). **RMSE** $\sqrt{\text{MSE}}$ (units). Errors 1, 1, 1, 9: MAE 3, RMSE 4.6.
- $R^2 = 1 - \text{SS}_{\text{res}} / \text{SS}_{\text{tot}}$ (can be negative out of sample). **MAPE** = mean |error|/|actual| (explodes near 0). **SMAPE**, **RMSLE** (relative errors).
- **Ranking:** precision@K, recall@K, MRR (1/rank of first hit), MAP, **NDCG** (graded relevance, discounts by log2 position).
- **Clustering:** silhouette (-1 to 1: closeness to own vs nearest cluster), Davies-Bouldin (lower better), inertia/elbow; with labels: adjusted Rand index, NMI. Judge usefulness by actionability.

IMBALANCED DATA PLAYBOOK

1. **Metrics first:** precision, recall, PR-AUC; never plain accuracy.
2. **Stratified** splits so every fold has positives.
3. **Class weights** (`class_weight="balanced"`, `scale_pos_weight`): the simplest fix.

4. **Resample training folds only:** undersample the majority, oversample the minority, **SMOTE** (synthetic neighbours), ADASYN.
5. **Tune the threshold** on a validation set; recalculate probabilities if resampled.
6. Try anomaly detection when positives are extremely rare; collect more positives; cost-sensitive learning.

TRAP Resampling before the split duplicates rows into validation and inflates scores. Do it inside the training fold (`imblearn.Pipeline`).

```
from sklearn.metrics import (
    precision_recall_curve,
    classification_report,
    average_precision_score)
p, r, t = precision_recall_curve(y, score)
print(classification_report(y, pred))
```

THRESHOLD AND CALIBRATION RECIPE

1. Get scores on a **validation** set (never the training set).
2. Compute precision/recall (or cost) for each threshold; plot the PR curve.
3. Choose the threshold that meets the business constraint (recall $\geq 80\%$, review capacity, minimum expected cost).
4. Fix it, then report the test-set metrics once at that threshold.
5. Re-check calibration after resampling or drift; recalibrate (Platt, isotonic).

METRIC TRAPS

TRAP Optimising AUC when only the top 1% is actioned; use precision@K. Comparing models by different thresholds or on different data. Reporting accuracy for imbalanced classes, R^2 as accuracy, or RMSE without a baseline (always compare with predicting the mean/last value). Tuning on the test set; macro vs micro vs weighted averaging in multiclass (macro treats classes equally).

SAY IT IN 20 SECONDS

- "Accuracy lies on imbalanced data; use precision, recall and PR-AUC."
- "Pick the threshold from the cost of each kind of error."
- "ROC-AUC is about ranking, not about calibrated probabilities."
- "Scale, impute and encode inside the pipeline, fit on train only."

Time Series, Clustering, Recommenders, Anomaly Detection

Forecasting, segmentation, recommendation and outlier detection: methods, parameters, metrics and the traps in each.

DS Prep
INTERVIEW CHEAT
SHEET

TIME SERIES BASICS

- Components: **trend**, **seasonality** (fixed period), **cycle**, **noise**. Additive: $y = T + S + e$; multiplicative: $y = T \times S \times e$ (seasonal swing grows with level; take logs).
- **Stationary**: constant mean, variance and autocovariance. Make stationary by differencing ($y_t - y_{t-1}$), seasonal differencing (lag m), log transform. Test with **ADF** (null: non-stationary) / KPSS.
- **ACF/PACF** plots: ACF cuts off at $q \rightarrow$ MA(q); PACF cuts off at $p \rightarrow$ AR(p). Autocorrelation at lag 7 = weekly pattern.
- **Always beat a baseline**: naive (last value), seasonal naive (same weekday last week), moving average.

SMOOTHING FORMULAS AND WORKFLOW

- **Simple exponential smoothing**: $\text{level}_t = \alpha y_t + (1 - \alpha) \text{level}_{t-1}$; α near 1 reacts fast, near 0 is smooth. Holt adds a trend term, Holt-Winters a seasonal term.
- Moving average of window w : lags the series by about $w/2$.
- Workflow: plot $\rightarrow$ decompose (STL) $\rightarrow$ baseline $\rightarrow$ stationarity check $\rightarrow$ model $\rightarrow$ **residuals should look like white noise** (Ljung-Box test, no ACF spikes) $\rightarrow$ forecast with intervals $\rightarrow$ backtest.
- Missing timestamps: reindex to a regular calendar, fill (interpolate for sensors, 0 for counts), flag outliers, mind time zones and daylight saving.
- Many series: global model (one GBM across series with series-id features) often beats one ARIMA per series. Intermittent demand (many zeros): Croston, or count models.

FORECASTING MODELS

Model	Idea	Use
Moving average / exponential smoothing	weighted recent history	short horizon, level only
Holt-Winters (ETS)	level + trend + season	seasonal business series
ARIMA(p, d, q)	AR(p) past values, d differences, MA(q) past errors	single stationary-ish series
SARIMA	ARIMA + seasonal (P, D, Q, m)	seasonality
SARIMAX / regression	add external regressors	promotions, price, weather
Prophet	trend + seasonality + holidays	business series with gaps, holidays
GBM / LightGBM on lag features	lags, rolling stats, calendar	many series, rich features
LSTM / Transformer	sequence nets	long histories, many series

- Feature recipe: lags (1, 7, 28), rolling mean/std, expanding stats, differences, day of week, month, holiday, price/promo. Use only information known at forecast time.

VALIDATION AND METRICS

- **Never random-split**. Use a temporal holdout and **rolling-origin / forward-chaining** CV (`TimeSeriesSplit`); gap between train and test if features look ahead.
- Metrics: **MAE**, **RMSE**, **MAPE** (breaks near 0), **sMAPE**, **MASE** (vs naive; < 1 beats naive). Evaluate by horizon; add prediction intervals.

- Pitfalls: leakage from future data (rolling windows computed centred), ignoring holidays/structural breaks, using ARIMA on non-stationary data, forecasting too far ahead, tuning on one split.

```
from statsmodels.tsa.arima.model import ARIMA
m = ARIMA(y, order=(1, 1, 1)).fit()
m.forecast(steps=7)
from statsmodels.tsa.stattools import adfuller
adfuller(y)[1] # p < 0.05: stationary
from sklearn.model_selection import TimeSeriesSplit
```

CLUSTERING

- Unsupervised grouping; **scale features first**; no ground truth, so judge by metrics and usefulness.

Method	Idea and limits
K-means	assign to nearest centre, recompute means, repeat. Fast; needs k , assumes round similar-size clusters, sensitive to scale, outliers and init (use <code>k-means++</code>)
Hierarchical	merge closest groups, cut the dendrogram. No k up front; $O(n^2)$; linkage choice (Ward, single, complete)
DBSCAN	dense regions via <code>eps</code> , <code>min_samples</code> ; noise = -1. Arbitrary shapes, finds outliers; struggles with varying density
GMM	soft assignments with Gaussians (EM). Elliptical clusters, probabilities; needs k

- **Choose k** : elbow of inertia (WCSS), **silhouette** score (-1 to 1), gap statistic, stability across seeds, business sense.
- Mixed data: Gower distance / k -prototypes. High dimensions: PCA first. Evaluate with labels (ARI, NMI) only if available.
- Profile clusters (means per feature, sizes), name them, test they are stable and actionable. K-means distance is Euclidean: standardise.

RECOMMENDER SYSTEMS

- **Content-based:** recommend items similar to what the user liked (item features, TF-IDF, embeddings, cosine similarity). No cold start for items; narrow, filter-bubble risk.
- **Collaborative filtering:** use behaviour. **User-user / item-item** neighbourhoods (cosine, Pearson); **matrix factorisation** (ALS, SVD): $R \sim U \text{ times } V\text{-transpose}$, learn k -dim vectors so the dot product predicts the rating; implicit feedback (clicks, views) needs confidence weighting.
- **Hybrid** and **deep** models (two-tower, neural CF, sequence models) combine signals.
- **Two stages at scale:** candidate generation (retrieve hundreds from millions with approximate nearest neighbours) then a **ranking** model (GBM / NN using user, item, context features), then re-ranking (diversity, business rules).
- **Cold start:** new user: popularity, context (location, device), onboarding questions; new item: content features, exploration.
- **Metrics:** offline precision@K, recall@K, MAP, **NDCG@K**, hit rate, coverage, diversity, novelty; RMSE only for explicit ratings. Online: A/B on CTR, conversion, retention, revenue.
- **Pitfalls:** popularity bias, feedback loops, position bias in clicks, offline-online mismatch, sparsity, leakage (use time-based splits), implicit data has no negatives (sample them).

ANOMALY DETECTION

- Define "anomalous" for the business first: point (one odd value), contextual (odd for the time/segment), collective (odd pattern).

Method	Idea	Note
z-score	$\text{abs}(x - \text{mean}) / \text{SD} > 3$	assumes normal; outliers inflate mean and SD
IQR rule	outside $Q1 - 1.5 \text{ IQR}$, $Q3 + 1.5 \text{ IQR}$	robust, simple
Median/MAD	robust $z = 0.6745 (x - \text{median}) / \text{MAD}$	skewed data
Isolation Forest	random splits isolate outliers in few steps	high dimensions, fast
LOF	density versus neighbours	local anomalies
One-class SVM	boundary around normal data	small data, scale needed
Autoencoder	high reconstruction error = odd	complex data, NN
Time series	residuals from a forecast, seasonal decomposition (STL)	check against the pattern

- Example: orders 98-103 normal, one day 180: mean 108.9, SD 26.7 $\rightarrow z = 2.7$ (missed); IQR fence 108.5 $\rightarrow$ flagged.
- Evaluation with few labels: precision@K by expert review, injected synthetic anomalies, alert fatigue (false alarms per day). Handle drift: what is normal changes; retrain, use rolling baselines.
- Supervised fraud: imbalance playbook (Module 8) + rules + model.

```
from sklearn.ensemble import
IsolationForest
m = IsolationForest(contamination=0.01,
random_state=0).fit(X)
flag = m.predict(X) == -1      #
anomalies
score = -m.score_samples(X)   # higher
= odder
```

SIMILARITY AND ASSOCIATION RULES

Measure	Use
Cosine similarity	direction of vectors (text, ratings); ignores magnitude
Pearson correlation	rating similarity after centring users
Jaccard	overlap of sets (items bought)
Euclidean	dense numeric features (scale first)

- **Market basket / association rules:** support = $P(A \text{ and } B)$; confidence = $P(B | A)$; **lift** = confidence/ $P(B)$ (> 1 means positive association). Apriori / FP-growth. Beware: popular items give high confidence but lift ~ 1 .
- Evaluate recommenders offline with a **time-based split** (train on the past, test on later interactions), then confirm with an A/B test: offline gains often vanish online.
- **Exploration:** bandits (epsilon-greedy, Thompson) to learn about new items; avoid showing only what is already popular.

TIME SERIES ANOMALIES AND MONITORING

- Forecast, then flag residuals beyond ~ 3 SD or beyond the prediction interval. Use STL (seasonal-trend-loess) to remove trend and seasonality, then apply z/IQR rules to the remainder.
- Alerting: tune the threshold for false alarms per day; use rolling windows; compare with the same weekday last week.
- Drift in what is "normal" (new product launch, holiday) causes alert storms: allow manual overrides and retrain.

METHOD CHOOSER

Task	Start with
Forecast a daily metric	seasonal naive $\rightarrow$ Holt-Winters/Prophet $\rightarrow$ GBM with lags
Customer segments	K-means on scaled RFM features; check silhouette
Odd-shaped clusters / noise	DBSCAN
"Customers like you also bought"	item-item CF, then factorisation
New users, little data	popularity + content
Detect fraud with few labels	rules + Isolation Forest, review top-K

SAY IT IN 20 SECONDS

- "Time series: respect order, beat a naive baseline, validate forward in time."
- "K-means needs scaled features and a chosen k ; DBSCAN finds odd shapes and noise."
- "Recommenders retrieve, then rank; cold start needs content and popularity."
- "A z-score can miss an outlier because the outlier inflates the SD."

NEURAL NETWORK MECHANICS

- A **neuron**: output = activation($w \cdot x + b$). Layers of neurons; the **non-linear activation** is what lets the network learn curves (stacked linear layers collapse to one linear map).
- Training loop**: forward pass → loss → **backpropagation** (chain rule gives $d\text{Loss}/dw$ for every weight) → optimiser step (Adam/SGD) → repeat per mini-batch; one pass over the data = epoch.

Activation	Formula / range	Note
ReLU	$\max(0, x)$	default hidden; "dying ReLU"
Leaky ReLU / GELU	small slope for $x < 0$ / smooth	transformers use GELU
Sigmoid	$1/(1 + e^{-(x)})$, (0, 1)	binary output; vanishing gradients
Tanh	(-1, 1)	zero-centred; RNNs
Softmax	$e^{(z_i)} / \sum e^{(z_j)}$	multiclass probabilities

- Output + loss pairs**: regression: linear + MSE; binary: sigmoid + binary cross-entropy; multiclass: softmax + cross-entropy.
- Optimisers**: SGD + momentum; **Adam** (default); learning-rate schedules, warm-up; typical Adam lr $1e-3$ ($1e-4$ for fine-tuning).
- Initialisation**: Xavier/Glorot (sigmoid/tanh), He (ReLU). **Vanishing/exploding gradients**: ReLU, good init, **batch/layer norm**, residual connections, gradient clipping.
- Regularise**: dropout (randomly zero units while training), weight decay (L2), early stopping, data augmentation, more data, smaller net.
- Batch norm** normalises activations per batch (faster, stabler); **layer norm** per example (transformers).
- For small tabular data, gradient boosting usually beats deep nets.

ARCHITECTURES AT A GLANCE

Type	Good for	Core idea
MLP	tabular, simple	dense layers
CNN	images, audio	local filters with shared weights, pooling
RNN / LSTM / GRU	sequences	hidden state; LSTM gates fight vanishing gradients
Transformer	text, now vision	self-attention, parallel
Autoencoder / VAE	compression, anomalies, generation	encoder → latent → decoder
GAN / diffusion	image generation	adversarial / denoising

- CNN output size**: $(W - K + 2P)/S + 1$. Parameters of a conv layer: $K \cdot K \cdot C_{in} \cdot C_{out} + C_{out}$.
- Framework skeleton (PyTorch): `loss = crit(model(x), y); opt.zero_grad(); loss.backward(); opt.step(); model.eval() and torch.no_grad() for inference.`

BACKPROP AND SEQUENCE MODELS BY HAND

- One neuron, squared loss: $y_{\text{hat}} = \sigma(wx + b)$, $L = (y_{\text{hat}} - y)^2$; $dL/dw = 2(y_{\text{hat}} - y) \cdot \sigma'(z) \cdot x$; update $w \leftarrow w - \eta \cdot dL/dw$. Backprop reuses each layer's gradient going backwards.
- RNN**: $h_t = \tanh(W \cdot h_{t-1} + U \cdot x_t)$; gradients shrink over long sequences. **LSTM** adds forget, input and output gates and a cell state; **GRU** is a lighter two-gate version.
- Parameter count of a dense layer: $n_{in} \cdot n_{out} + n_{out}$.
- Hyperparameters**: learning rate (most important), batch size, depth/width, dropout rate, weight decay, epochs with early stopping.
- Debug order: data and labels → loss at start ($\sim \ln(K)$ for K classes) → overfit one batch → learning rate → architecture.

COMPUTER VISION

- Convolution** slides small filters (edges → textures → parts → objects); pooling shrinks maps. Far fewer parameters than dense layers.
- Tasks: **classification** (label), **detection** (boxes + labels: YOLO, Faster R-CNN), **segmentation** (label per pixel: U-Net, Mask R-CNN).
- Transfer learning**: take a model pretrained on ImageNet, replace the head, freeze early layers, fine-tune the rest. Works with small datasets. **Augmentation**: flips, crops, colour jitter.
- IoU** = overlap area/union area; 2 boxes of area 16 overlapping 8: $8/24 = 0.33$. A detection counts correct at $\text{IoU} \geq 0.5$. **AP** = area under the precision-recall curve per class; **mAP** = mean over classes (and IoU thresholds).
- Check train and production images look alike (lighting, devices).

CLASSICAL NLP PIPELINE

- Clean → tokenise → (lowercase, stop words, stem/lemmatise) → vectorise → model.
- Bag of words**: counts. **TF-IDF** = $\text{tf} \times \log(N/\text{df})$: frequent in this document, rare overall; sparse, ignores order and meaning. n-grams capture short phrases.
- Embeddings**: dense vectors where similar meanings are close (word2vec, GloVe, fastText); **contextual** embeddings (BERT) give "bank" different vectors by sentence. Cosine similarity compares vectors.
- Baselines: TF-IDF + logistic regression / linear SVM / Naive Bayes are strong and cheap; then fine-tune a transformer.
- Tasks and metrics: classification (F1), NER, summarisation (ROUGE), translation (BLEU), retrieval (recall@K, MRR), language modelling (perplexity).

```
from sklearn.feature_extraction.text
import (
    TfidfVectorizer)
v = TfidfVectorizer(ngram_range=(1, 2),
    min_df=2, stop_words="english")
X = v.fit_transform(texts)
```

TRANSFORMERS AND ATTENTION

- **Self-attention:** each token makes a query Q, key K and value V. Attention = $\text{softmax}(Q \cdot K^T / \sqrt{d_k}) \cdot V$: every token takes a weighted average of all tokens' values, weights from query-key match. The $\sqrt{d_k}$ scaling keeps softmax from saturating.
- **Multi-head:** several attentions in parallel learn different relations. **Positional encodings** inject order. **Residual + layer norm**, feed-forward block per layer.
- **Encoder** (BERT: understanding, classification, embeddings), **decoder** (GPT: next-token generation, causal mask), encoder-decoder (T5: translation, summarisation).
- vs RNNs: parallel training, direct long-range links. **Cost grows as n^2 with sequence length**, which limits context.
- Pretrain on huge text → **fine-tune** for a task.

LLM FUNDAMENTALS

- **Tokens:** sub-word pieces (~0.75 words each); context window = max tokens in prompt + output. Cost and latency scale with tokens.
- Generation: the model outputs a probability over the next token. **Temperature 0** = near-deterministic, higher = more varied; **top-p** (nucleus) samples from the most likely mass; **top-k**.
- Training stages: pretraining (next token) → **instruction tuning (SFT)** → **RLHF / preference tuning (DPO)**.
- **Prompting:** clear role, task and format; **few-shot** examples; ask for step-by-step reasoning (chain of thought); delimit inputs; request JSON and validate it; keep prompts versioned and tested.
- **Hallucination:** fluent but false output. Reduce with retrieval (RAG) and citations, lower temperature, constrained formats, verification/tool calls, "say you do not know".
- **Evaluation:** a golden set of real questions; automatic checks, LLM-as-judge (with human calibration); track regressions per prompt/model change.
- Safety: prompt injection (untrusted text containing instructions), PII leakage, jailbreaks: filter, isolate untrusted content, least-privilege tools.

RAG END TO END

- **Offline:** load → clean → **chunk** (200-800 tokens, overlap 10-20%, respect headings) → **embed** → store in a vector index (FAISS, pgvector, Pinecone) with metadata.
- **Online:** embed query → retrieve top-k (similarity: cosine/dot; ANN indexes: HNSW, IVF) → optional **rerank** (cross-encoder) → build prompt with sources and "answer only from the context" → generate with citations.
- Improve: **hybrid search** (BM25 keyword + vectors), metadata filters, query rewriting, parent-document retrieval, good chunking.
- **Evaluate separately:** retrieval (recall@K, hit rate, MRR) and generation (faithfulness/groundedness, answer relevance, correctness). If the answer is wrong, **check retrieval first**.
- Failure modes: wrong chunk size, stale index, missing documents, embedding model mismatch, context overflow, lost-in-the-middle.

RAG VS FINE-TUNING, AGENTS

Need	Use
Fresh, private or changing facts, citations	RAG
Style, tone, format, specialised behaviour, lower latency	Fine-tune (LoRA / QLoRA: train small low-rank adapters)
Both	RAG + a lightly fine-tuned model

- Order of effort: prompting → RAG → fine-tuning.
- **Agent:** an LLM that plans and calls tools (search, SQL, code, APIs) in a loop. ReAct pattern: reason → act → observe. Add step limits, logging, validation, human approval for risky actions; errors compound over steps.
- **Embedding models** for search vs **generation models:** different jobs. **Function calling / structured outputs** make tool use reliable.
- Project ideas: document Q&A bot, support-ticket triage, text-to-SQL assistant, resume screener, meeting summariser (with evaluation set and cost report).
- Cost levers: smaller model, caching, shorter prompts, batching, streaming.

TOKENISATION, EMBEDDINGS, PROMPTS

- **BPE / WordPiece** tokenisers split rare words into pieces: no out-of-vocabulary words; counts differ by language (non-English costs more tokens).
- Sentence embeddings (e.g. SBERT) → cosine similarity for semantic search, clustering, dedupe; normalise vectors so dot product = cosine.
- Prompt skeleton: **Role** (you are ...), **Task** (do ...), **Context** (use only ...), **Format** (JSON with keys ...), **Examples** (2-3), **Constraints** (if unknown say so).
- Eval table: question | expected | retrieved | answer | grounded? | correct? → aggregate; re-run after every prompt, chunking or model change.
- Rough sizes: context 8k-200k tokens; model parameter count ~ memory (7B params ~ 14 GB in fp16, ~4 GB with 4-bit quantisation).

TRAINING TIPS AND PITFALLS

- Overfit a tiny batch first to check the pipeline; watch train vs validation loss; stop when validation stalls.
- Loss NaN: lower the learning rate, check inputs, clip gradients. Class imbalance: weights or sampling.
- Data leakage via near-duplicates between train and test (images, text).
- GPU memory: smaller batch, mixed precision (fp16/bf16), gradient accumulation.
- Reproducibility: seeds, pinned versions, logged configs.

SAY IT IN 20 SECONDS

- "Attention lets each token take a weighted average of all tokens."
- "RAG adds knowledge at question time; fine-tuning changes behaviour."
- "If a RAG answer is wrong, check retrieval before the prompt."
- "ReLU gives non-linearity; backprop is the chain rule."
- "Transfer learning: pretrained backbone, new head, fine-tune."

DATA ENGINEERING CONCEPTS

- **ETL**: extract, transform, then load. **ELT**: load raw data first, transform inside the warehouse (common with cloud warehouses); keeps raw data for reprocessing.

Store	What	Use
Data lake	raw files in cheap object storage (S3, GCS)	all data, ML, schema-on-read
Data warehouse	modelled, columnar tables (BigQuery, Snowflake, Redshift)	BI, SQL analytics, schema-on-write
Lakehouse	lake + warehouse features (Delta, Iceberg, Hudi)	both

- **Batch** (nightly/hourly) vs **streaming** (events as they arrive: Kafka, Kinesis, Flink, Spark Structured Streaming). Choose batch unless latency truly matters.
- **Orchestration**: Airflow/Dagster/Prefect schedule tasks as a DAG with retries and alerts. **dbt** for SQL transformations with tests.
- **Good pipelines: idempotent** (re-running gives the same result), incremental loads, backfills, schema checks, data-quality tests (nulls, duplicates, freshness, row counts), monitoring and lineage.
- **File formats**: CSV/JSON (row text, slow), **Parquet/ORC** (columnar, compressed, fast scans, predicate pushdown), Avro (row, schema evolution, streaming). Partition by date; avoid tiny files.
- **Modelling**: star schema = fact table (events, measures) + dimension tables (who, what, when); **SCD type 2** keeps history of changing dimensions; normalised for OLTP, denormalised for analytics.
- **CDC** (change data capture) streams database changes; **data contracts** define schemas between teams.

PYSPARK

- Distributed processing: data split into **partitions** across executors; the driver plans work. **Lazy evaluation**: transformations (select, filter, join, groupBy) build a plan; **actions** (count, show, collect, write) run it.
- **Narrow** transformations stay within a partition; **wide** ones (join, groupBy, distinct, orderBy) cause a **shuffle** (network + disk): the main cost.

```
from pyspark.sql import SparkSession, functions as F
spark = SparkSession.builder.getOrCreate()
df = spark.read.parquet("s3://b/events/")
out = (df.filter(F.col("dt") >= "2025-01-01")
      .groupBy("user_id")
      .agg(F.count("*").alias("n"),
           F.sum("amt").alias("rev"))
      .join(F.broadcast(users), "user_id", "left"))
out.write.mode("overwrite").parquet("o/")
```

- **Speed**: filter and select early, **broadcast** the small side of a join, avoid wide ops you do not need, use built-in functions (not Python UDFs), cache reused data (df.cache()), repartition sensibly (coalesce to reduce), fix **skew** (one huge key: salting, broadcast, AQE).
- **collect()** pulls all data to the driver: can crash. DataFrames are optimised by Catalyst; RDDs are lower level.
- **Window**:

```
F.row_number().over(Window.partitionBy("g").orderBy("d"))
```

DATABASES

Type	Examples	Good for
Relational (SQL)	Postgres, MySQL	transactions, joins, ACID
Key-value	Redis, DynamoDB	fast lookups, caching, sessions
Document	MongoDB	flexible JSON
Wide-column	Cassandra, HBase	huge write volume
Graph	Neo4j	relationships
Columnar warehouse	BigQuery, Snowflake	analytic scans
Vector	pgvector, Pinecone, FAISS	embedding search

- **OLTP** many small transactions (row store); **OLAP** big aggregations (column store).
- **CAP**: under a network partition a distributed system must choose consistency or availability. **ACID** vs **BASE** (eventual consistency). Sharding splits data across machines; replication copies it for reliability.
- Choose by access pattern: transactions → relational; scans → warehouse; fast lookup → key-value; files → object storage.

CLOUD SERVICE MAP

- Format: need = **AWS / GCP / Azure**.
- Object storage = S3 / Cloud Storage / Blob Storage.
- Warehouse = Redshift / BigQuery / Synapse.
- Managed Spark = EMR, Glue / Dataproc / Databricks.
- ML platform = SageMaker / Vertex AI / Azure ML.
- Serverless functions = Lambda / Cloud Functions / Functions.
- Streaming = Kinesis, MSK / Pub/Sub / Event Hubs.
- Orchestration = MWAA, Step Functions / Composer / Data Factory.
- IAM = identities and permissions (least privilege). Cost levers: right-size, spot instances, partition and prune, storage tiers, turn clusters off.

GIT

```
git clone URL; git status; git diff
git add -p; git commit -m "msg"
git switch -c feature/x
git push -u origin feature/x
git pull --rebase; git fetch
git merge main; git rebase main
git log --oneline --graph; git blame f
git stash; git stash pop
git restore f; git reset --soft HEAD~1
git revert SHA # safe undo on
shared branch
git cherry-pick SHA; git tag v1.0
```

- Branch → small commits → push → pull request → review → merge. **Merge** keeps history with a merge commit; **rebase** replays your commits for a linear history: never rebase shared branches.
- Conflict: edit markers <<<<<<, pick the right content, git add, continue.
.gitignore for data, secrets, .venv. A committed secret is leaked: **rotate it**, then remove it.

LINUX AND SHELL

```
ls -lah; cd; pwd; mkdir -p d; rm -r d
head -n 5 f; tail -f log; less f; wc -l f
grep -rn "err" . | sort | uniq -c | sort -nr
cut -d, -f1,3 f.csv; awk -F, '{s+=$2} END{print s}' f
find . -name "*.py"; du -sh *; df -h
chmod +x run.sh; ps aux | grep py; kill -9 PID
cat a.txt | tr a-z A-Z > b.txt; command >> log 2>&1
ssh user@host; scp f host:/path; curl -O URL
export VAR=1; env; which python; cron: 0 2 * * *
```

Pipes | chain commands; > overwrites, >> appends, 2>&1 merges errors.

ENVIRONMENTS AND DOCKER

```
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt; pip freeze > requirements.txt
conda create -n x python=3.11; conda activate x
docker build -t app .; docker run -p 8000:8000 app
docker ps; docker logs -f ID; docker exec -it ID sh
docker compose up -d
```

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
CMD ["uvicorn", "main:app", "--host", "0.0.0.0"]
```

- Image** = blueprint; **container** = running instance. Copy requirements first to use layer caching. Same image runs everywhere: reproducibility. Keep secrets out of images (env vars/secret managers). Pin versions, set random seeds.

BI TOOLS AND DASHBOARDS

- Tableau / Power BI / Looker** connect to data, build charts, filters, calculated fields, and dashboards. Know: measures vs dimensions, extracts vs live connections, LOD expressions/DAX basics.
- A good dashboard answers specific questions: key metric first, trend, breakdown, filters, consistent definitions, last-refreshed date, few colours.
- Ask who decides what from it; agree metric definitions; avoid vanity metrics.

EXCEL ESSENTIALS

- Lookups: XLOOKUP(key, keys, vals) (or VLOOKUP(key, table, col, FALSE); INDEX/MATCH). #N/A = key missing, type mismatch or stray space (TRIM).
- Aggregation: SUMIFS, COUNTIFS, AVERAGEIFS; logic IF, IFS, IFERROR; text LEFT/MID/RIGHT, TEXTJOIN, SPLIT via Text-to-columns; dates EOMONTH, DATEDIF, TEXT(d, "yyyy-mm").
- Pivot tables** = GROUP BY (rows = groups, values = aggregates, columns = pivot); slicers filter; refresh after data changes. Lookup = left join.
- Tips: absolute references \$A\$1, named ranges, Ctrl+Shift+arrows, data validation, conditional formatting, Power Query for repeatable cleaning.

PIPELINES IN PRACTICE

- Incremental load / upsert:** MERGE INTO tgt USING src ON key WHEN MATCHED THEN UPDATE ... WHEN NOT MATCHED THEN INSERT ...; keep a **watermark** (last processed timestamp); make reruns safe with dedupe on a key.

- Streaming:** event time vs processing time; **windows** (tumbling, sliding, session); **watermarks** for late data; delivery semantics at-most-once, at-least-once, **exactly-once** (idempotent writes + transactions). Kafka: topics, partitions, consumer groups, offsets.
- Data quality checks:** not-null, unique keys, accepted values, referential integrity, freshness, row-count and distribution drift (Great Expectations, dbt tests).

```
SELECT COUNT(*) - COUNT(DISTINCT id) AS
dups,
SUM(CASE WHEN amt IS NULL THEN 1
ELSE 0 END) AS nulls,
MAX(loaded_at) AS freshness
FROM fact_orders;
```

- APIs and JSON:** REST verbs GET/POST/PUT/DELETE, status codes (200 ok, 400 client error, 401/403 auth, 404, 429 rate limit, 500 server error); requests.get(url, timeout=10).json(); paginate and retry with backoff.
- CI/CD:** run tests and linters on every push (GitHub Actions); build the image; deploy automatically to staging, then production.
- Testing:** unit tests for transformations, sample-data integration tests, schema tests.

COST AND PERFORMANCE HABITS

- Store as Parquet, partition by date, prune partitions in queries, avoid SELECT * on columnar warehouses (pay per bytes scanned).
- Right-size clusters, auto-terminate idle ones, use spot capacity for batch, cache hot results.
- Monitor job duration, failures, data freshness, and cost per run.

SAY IT IN 20 SECONDS

- "ELT loads raw first and transforms in the warehouse."
- "Spark is lazy; shuffles are the cost; broadcast the small table."
- "Choose the database by access pattern: transactions, scans, lookups."
- "Merge keeps history, rebase rewrites it; never rebase shared branches."
- "Docker bundles code and environment so it runs the same everywhere."

THE ML LIFECYCLE

- 1. **Frame** the problem: decision, metric, cost of errors, constraints (latency, explainability).
 - 2. **Data**: collect, label, validate, version.
 - 3. **Features** and a **baseline**; train and evaluate with the right split.
 - 4. **Package and deploy**: batch job or online service.
 - 5. **Monitor**: system health, data quality, drift, model performance, business impact.
 - 6. **Retrain / improve** and release safely; **retire** old models.
- It is a loop: monitoring sends you back to data and features. Most effort is data and operations, not model code. A model in a notebook has no value until it is deployed and monitored.
 - **Version everything**: code (Git), data (DVC, lakeFS, snapshots), features, model artefacts and config, environment (Docker, pinned deps) so any prediction can be reproduced.

SERVING PATTERNS

Pattern	How	Use when
Batch	score on a schedule, store results	recommendations refreshed daily, risk lists; simplest
Online (real-time)	API returns a prediction in ms	fraud at checkout, search ranking
Streaming	score events as they arrive	monitoring, alerts
Edge / on-device	model runs on phone or device	offline, privacy, latency

- Online serving: wrap the model in an API (FastAPI/Flask/BentoML/TorchServe), containerise with Docker, run behind a load balancer on Kubernetes or a managed endpoint (SageMaker, Vertex AI). Concerns: **latency** (p50/p95/p99), throughput, autoscaling, cold starts, timeouts, fallbacks, caching, batching requests, cost.
- Reduce latency: smaller model, distillation, quantisation, ONNX, precomputed features, GPU only if needed.

```
from fastapi import FastAPI
import joblib
app = FastAPI(); model =
joblib.load("m.pkl")
@app.post("/predict")
def predict(x: dict):
    p = model.predict_proba([[x["a"],
x["b"]]])
    return {"score": float(p[0][1])}
```

TRAINING-SERVING SKEW AND FEATURE STORES

- **Skew**: features computed differently in training (SQL on history) and serving (service code), or with data not available at serving time. Causes silent accuracy loss.
- Prevent: **one shared feature definition (feature store**: Feast, Tecton, Databricks) with an offline store for training and an online store for low-latency serving; point-in-time-correct joins; log served features and compare with training; same preprocessing code in a versioned pipeline artefact.
- Training data must reflect what was known **at prediction time** (no future data), including late-arriving labels.

DRIFT AND DECAY

Type	Meaning	Detect
Data (covariate) drift	input distribution P(X) changes	PSI, KS test, chi-square, summary stats
Label (prior) drift	outcome rate changes	label rate over time
Concept drift	relationship P(y given X) changes	performance drop with labels; error by segment
Upstream change	schema, units, missing rates, new categories	data-quality checks

- **PSI** = $\sum (\text{actual}\% - \text{expected}\%) \times \ln(\text{actual}\%/\text{expected}\%)$ per bucket. Rule of thumb: < 0.1 stable; 0.1-0.25 moderate shift; > 0.25 major shift. Example: shares 50/30/20 → 40/30/30 gives PSI = 0.063.
- Causes: seasonality, product changes, new users, fraud adapting, pipeline bugs. Models decay: plan for it.

MONITORING LAYERS

Layer	What to track
System	latency, errors, throughput, CPU/GPU, memory, cost
Data	schema, missing/null rate, ranges, new categories, freshness, volume
Model	prediction distribution, drift (PSI/KS), confidence, performance on labels
Business	conversion, revenue, complaints, fairness by group

- **Delayed labels** (fraud chargebacks, churn): monitor input and prediction drift and proxy metrics now; compute true performance when labels arrive (evaluate on matured cohorts).
- Alerts with owners and runbooks; dashboards; log inputs, outputs, model version and request id (with privacy controls); sample for human review.

SAFE RELEASE STRATEGIES

- **Shadow mode:** new model scores live traffic silently; compare with current.
- **Canary:** route a small share (1-5%) to the new model; watch metrics; ramp or **roll back**.
- **A/B test:** randomise users, measure the business metric; **interleaving** for ranking. Bandits for fast learning.
- **Blue/green:** two environments, switch traffic, instant rollback. Feature flags for kill switches.
- Always keep the previous model deployable. Validate a new model against the champion on a holdout **before** release (champion/challenger).

RETRAINING

- Triggers: schedule (weekly), drift alert, performance drop, new data volume, business change.
- Retrain pipeline: fetch data → validate - > features → train → evaluate vs champion → register → deploy via canary. Automated but gated.
- **Continuous training (CT)** with CI/CD: tests on code and data, build image, deploy staging, promote.
- Online learning is rarer; beware feedback loops (the model changes the data it later trains on) and label bias.

CI/CD FOR ML AND TESTING

- **CI:** unit tests (feature code), data-validation tests (schema, ranges, no leakage), model tests (beats baseline, no regression on slices, invariance/directional tests), integration test of the API.
- **CD:** build the image, run smoke tests, deploy to staging, promote to production with approval; infrastructure as code.
- Registry stages: **None → Staging → Production → Archived**; each model links to code commit, data version, metrics and owner (**MLflow Model Registry**, SageMaker, Vertex).

EXPERIMENT TRACKING AND REPRODUCIBILITY

```
import mlflow
with mlflow.start_run():
    mlflow.log_params({"lr": 0.1,
"depth": 6})
    mlflow.log_metric("auc", 0.87)
    mlflow.sklearn.log_model(model,
"model")
```

- Track parameters, metrics, artefacts, code version, data version, environment. Fix random seeds; pin dependencies; containerise.
- **Model card:** purpose, data, metrics by segment, limits, risks, owner. Data card for datasets.

KUBERNETES AND SCALING BASICS

- **Pod** = one or more containers; **Deployment** keeps N replicas and does rolling updates; **Service** gives a stable address; **Ingress** routes external traffic; **HPA** autoscales on CPU/QPS; **readiness/liveness probes** gate traffic and restart failures.
- Resource requests/limits (CPU, memory, GPU); node pools for GPU; model files from object storage or baked into the image.
- SLOs: availability (99.9%), latency (p95 < 200 ms), error budget. Load-test before launch; add timeouts, retries with backoff, circuit breakers, and a rule-based fallback.

LLMOPS (GENAI IN PRODUCTION)

- Version **prompts**, retrieval settings and model names like code; run an **evaluation set** on every change (faithfulness, correctness, format validity, safety).
- Monitor token usage, cost per request, latency, refusal and error rates, user feedback (thumbs), hallucination reports; log prompts and outputs with PII controls.

- **Guardrails:** input and output filters, schema validation, tool allow-lists, rate limits, prompt-injection defences; cache frequent answers; route easy queries to a smaller model.
- Vector index freshness and re-embedding when the embedding model changes.

RESPONSIBLE ML AND SECURITY

- **Fairness:** compare error rates and outcomes across groups; remove proxies carefully; document trade-offs. **Explainability:** SHAP, reason codes (regulated lending). **Privacy:** minimise data, anonymise, access control, consent; **adversarial** inputs and model extraction; prompt injection for LLMs.
- Human in the loop for high-stakes decisions; fallback rules when the model is down or uncertain.

INCIDENT RUNBOOK: ACCURACY DROPPED

1. Confirm it is real (metric definition, delayed labels, volume).
2. Check data quality and upstream changes (schema, null rates, units).
3. Compare feature and prediction distributions with training; find the shifted features.
4. Check recent releases (code, model, config) and segments.
5. Mitigate: roll back, fall back to rules, retrain; then fix the root cause and add the missing alert.

SAY IT IN 20 SECONDS

- "Batch is simple; online is for real-time needs."
- "Skew means training and serving compute features differently; a feature store fixes it."
- "Data drift is inputs changing; concept drift is the relationship changing."
- "Ship with shadow or canary and a rollback."
- "Labels arrive late: watch drift and proxies now, performance later."